Python Virtual Environments

Instructions for Creating a Python Virtual Environment and Installing Python Libraries

Version 1.0

Table of Contents

Creating and Activating a Python environment
Installing Python libraries within the environment
Listing all installed Python libraries within the environment
Exporting all installed Python libraries into a configuration file
Recreating the same Python virtual environment

The goal of this document is to instruct you on how to create your own Python virtual environment. Environments in Python allow you to delineate your Python-based projects with differing required Python libraries. This prevents conflict in versions of required libraries and allows for automatic dependency resolution between versions of the required libraries.

To achieve this goal, we have the following objectives:

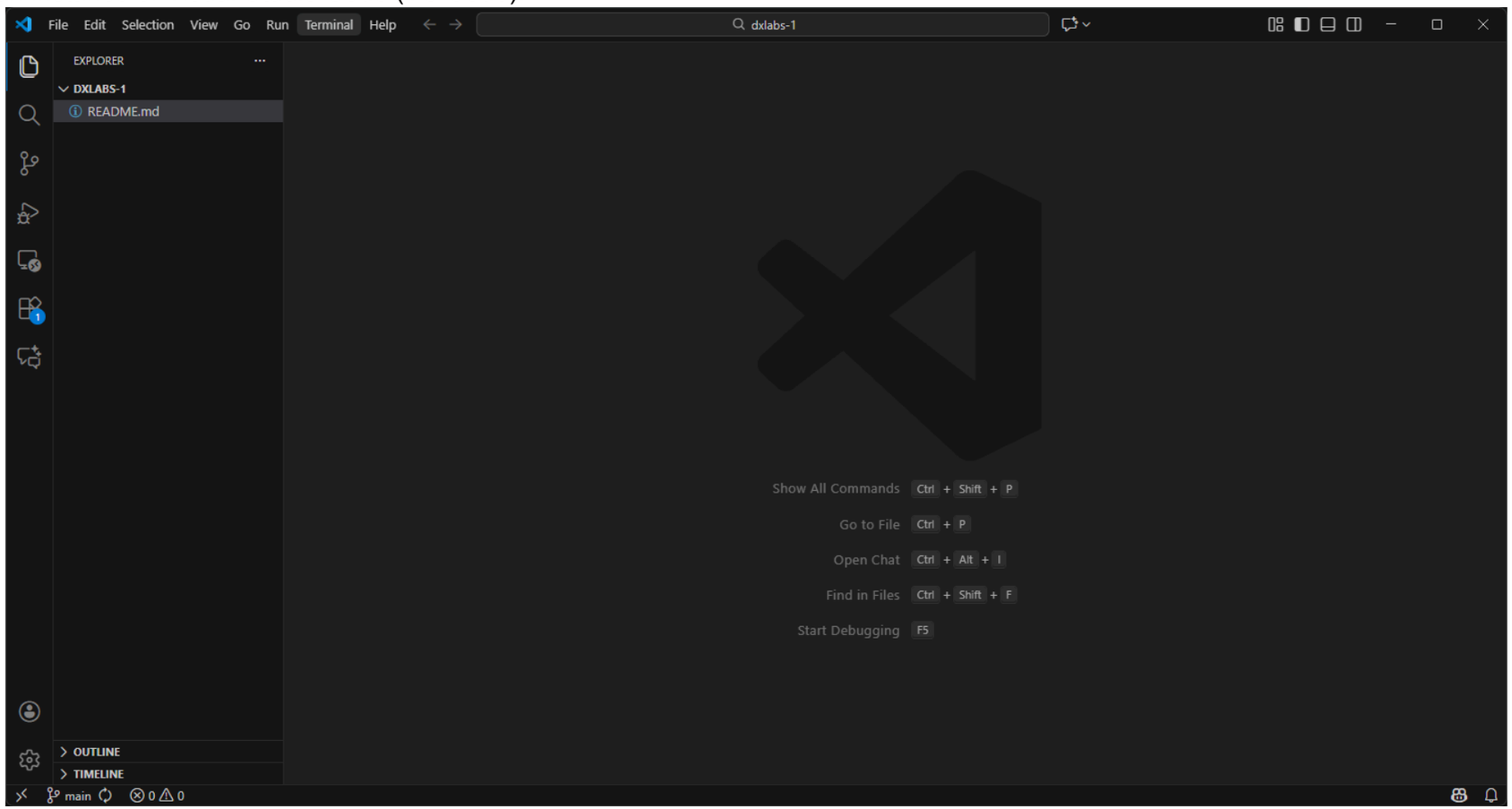
1. To instruct you on how to create your Python virtual environment.
2. To instruct you on how to install your required Python libraries within your created Python virtual environment.
3. To instruct you on how to list all the installed Python libraries within your Python virtual environment.

Prerequisites

1. Python 3.3 and up is installed in your computer.
2. Visual Studio Code is installed in your computer.

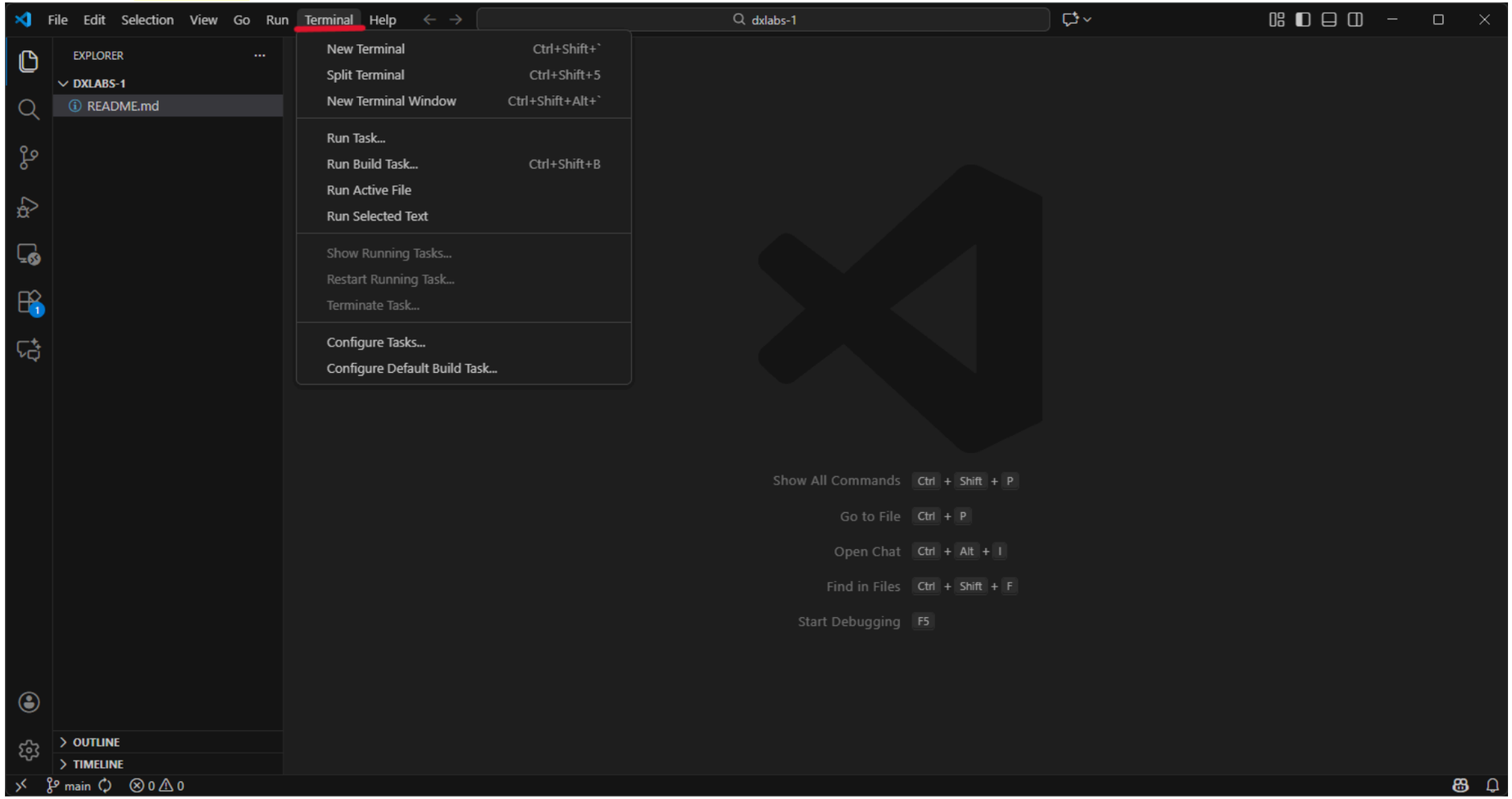
Creating and Activating a Python environment

1. Open an instance of Visual Studio Code (VS Code).

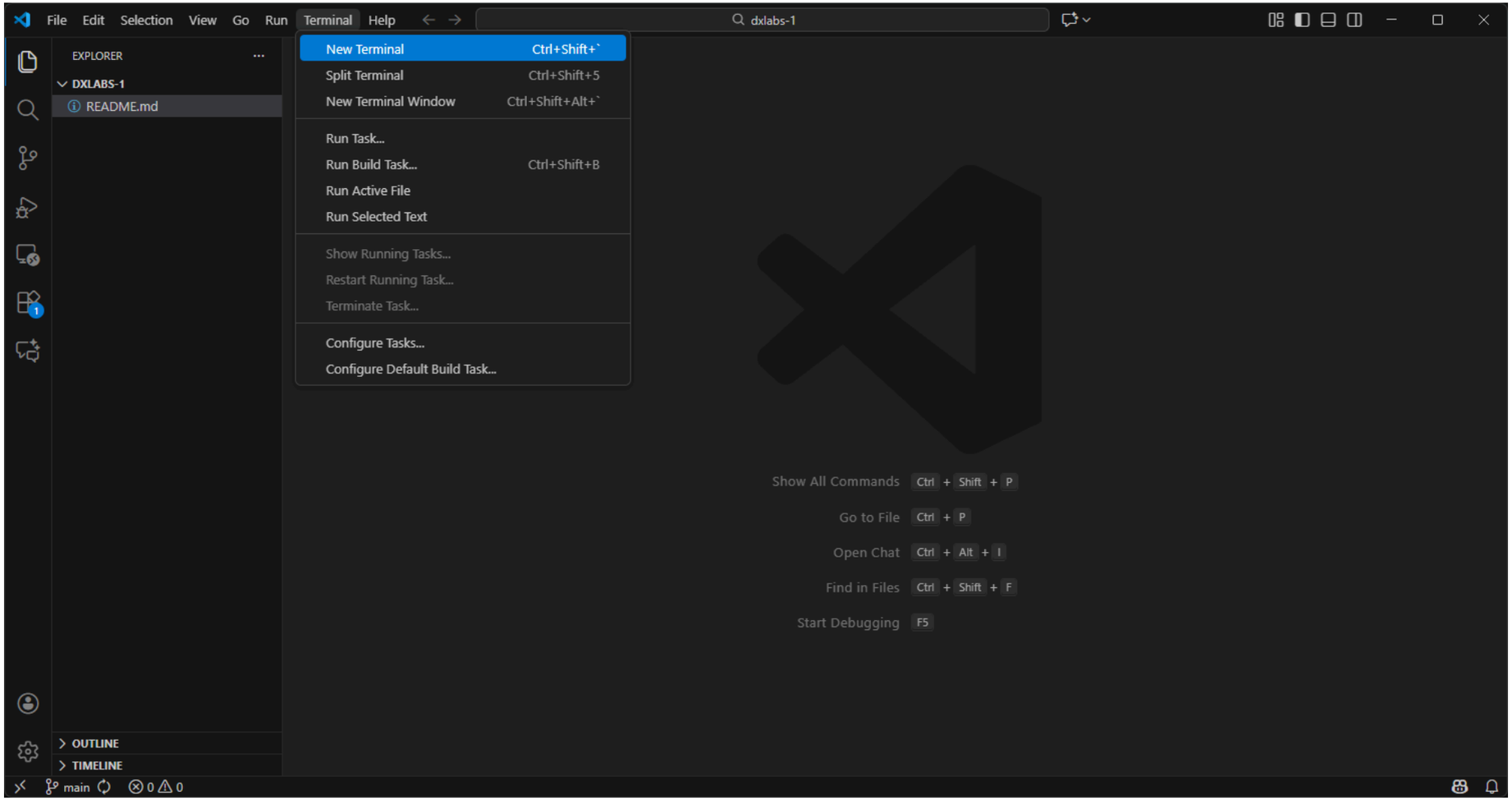


2. Open a command line interface in VS Code.

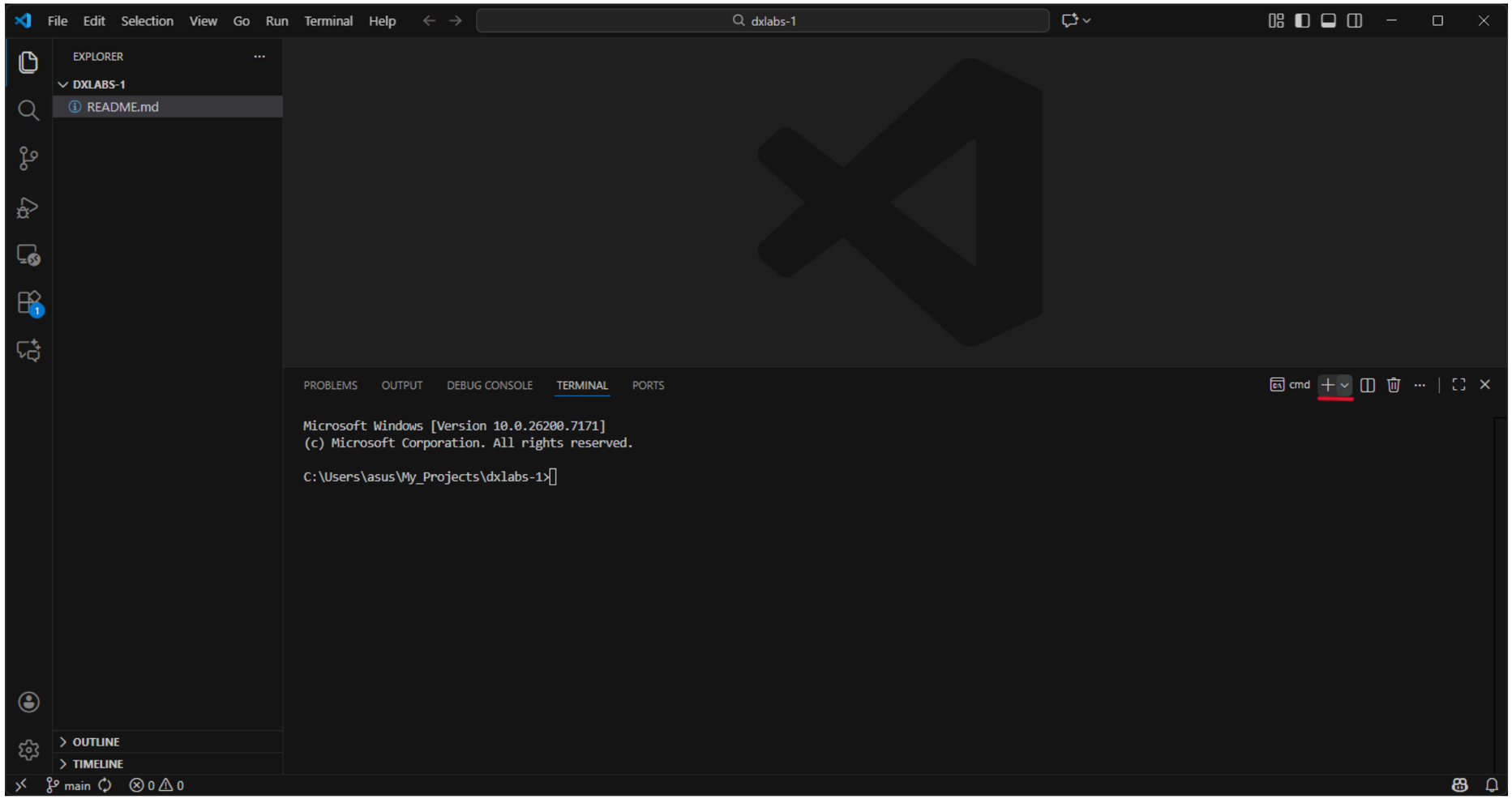
a. Click on the `Terminal` menu.



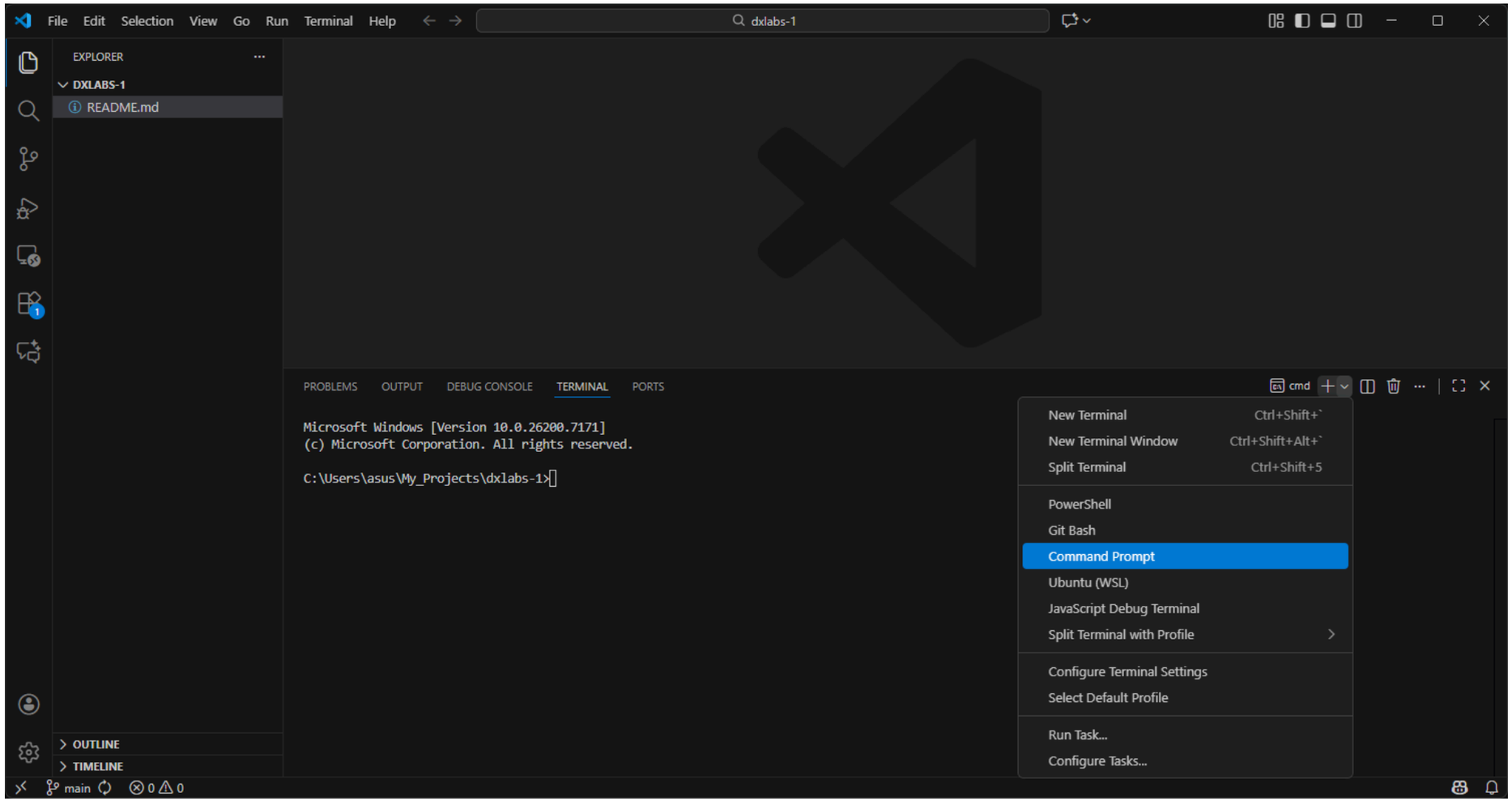
b. Click on the `New Terminal` menu item.



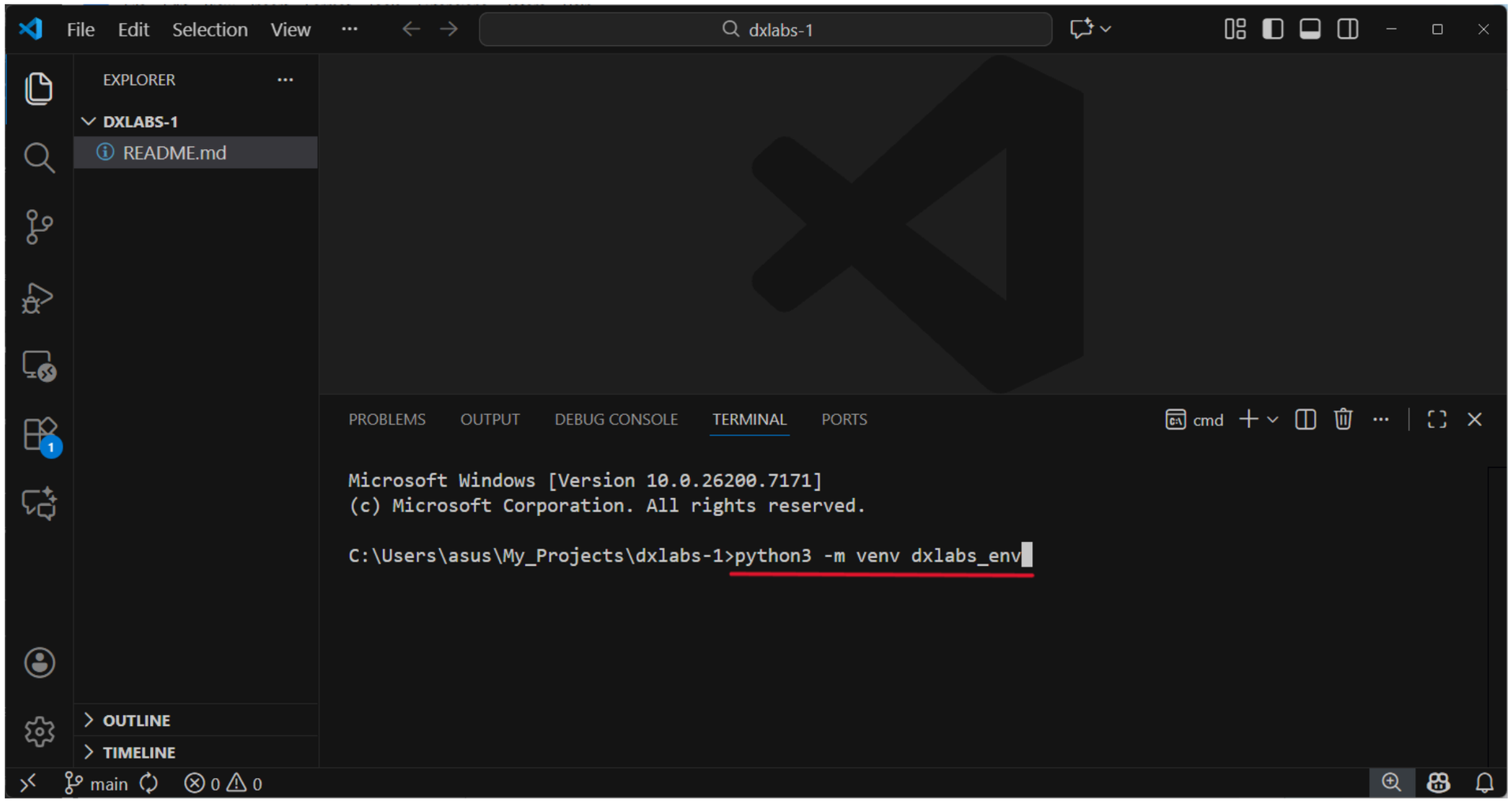
c. Note: If you happen to have opened a command line interface that is a Microsoft Windows PowerShell,
   i. Open a `Command Prompt` command line interface by clicking on the `Add Command Prompt` button.



ii. Click on the `Command Prompt` submenu item. A Command Prompt window will be opened in the Terminal View.



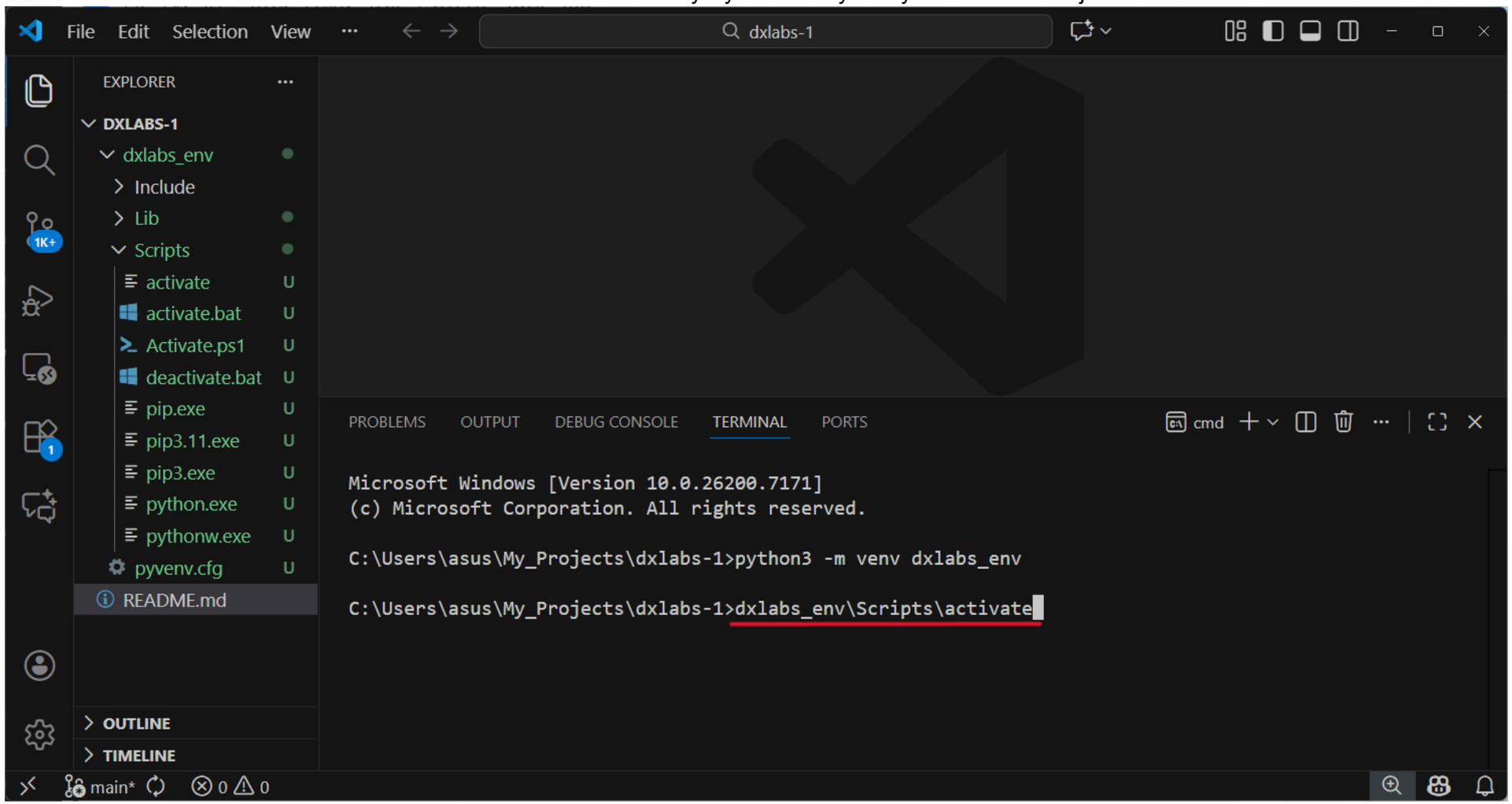
3. To create a Python virtual environment, use the following command prompt in your command line interface in VS Code:
    a. `python3 -m venv myenv`

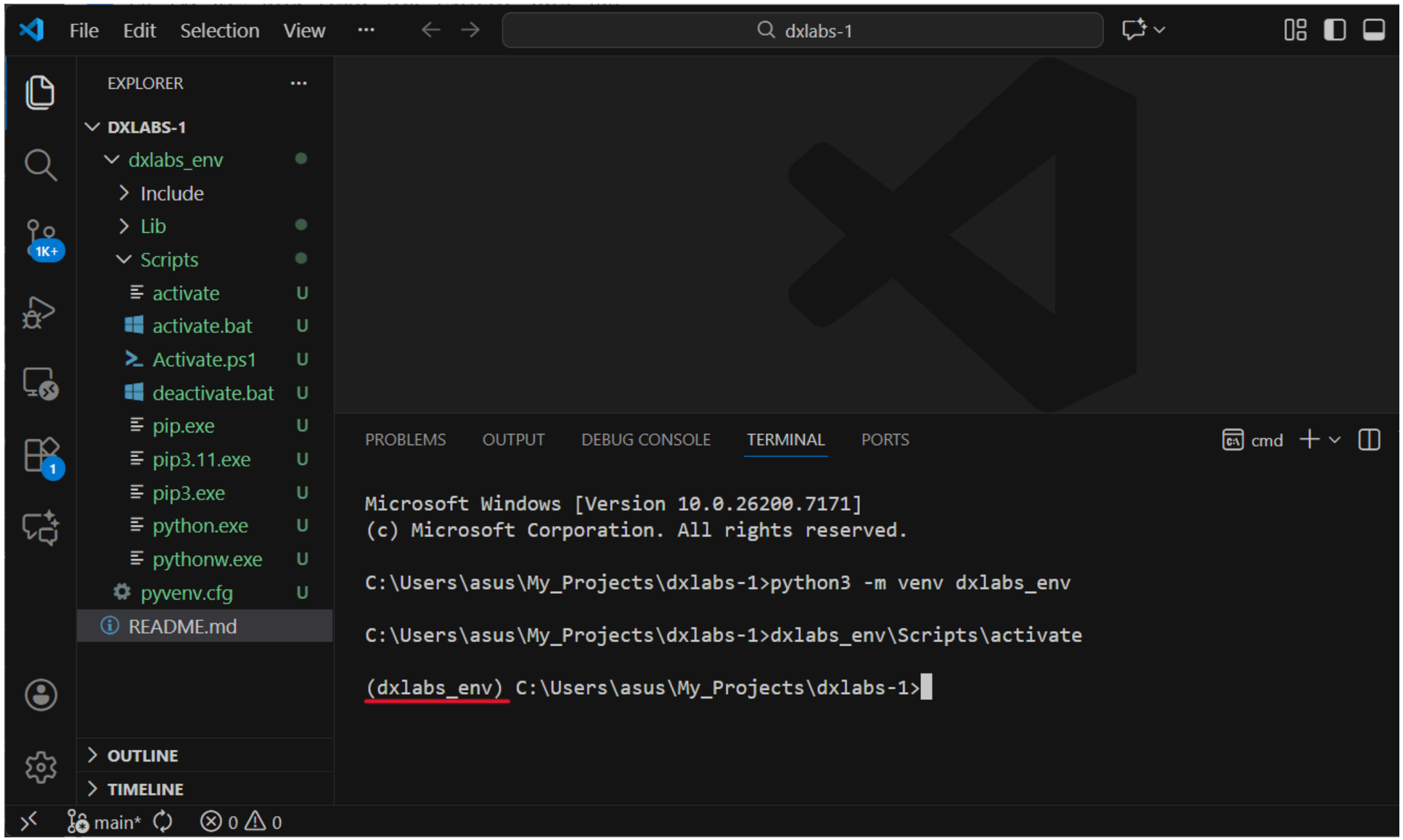


b. Note: Make sure to replace *myenv* with the name that you would like to give your Python virtual environment, e.g. `dxlabs_env`)

4. After creating the Python virtual environment, you will need to activate it to be able to use it for Python project management. To do so, use the following command in your command line interface in VS Code:

a. Windows: `myenv\Scripts\activate`. Your Python virtual environment will be activated. Any command that you issue in the command prompt will be executed within the context of this activated environment. Any Python library that you install will be just within the context of this environment.

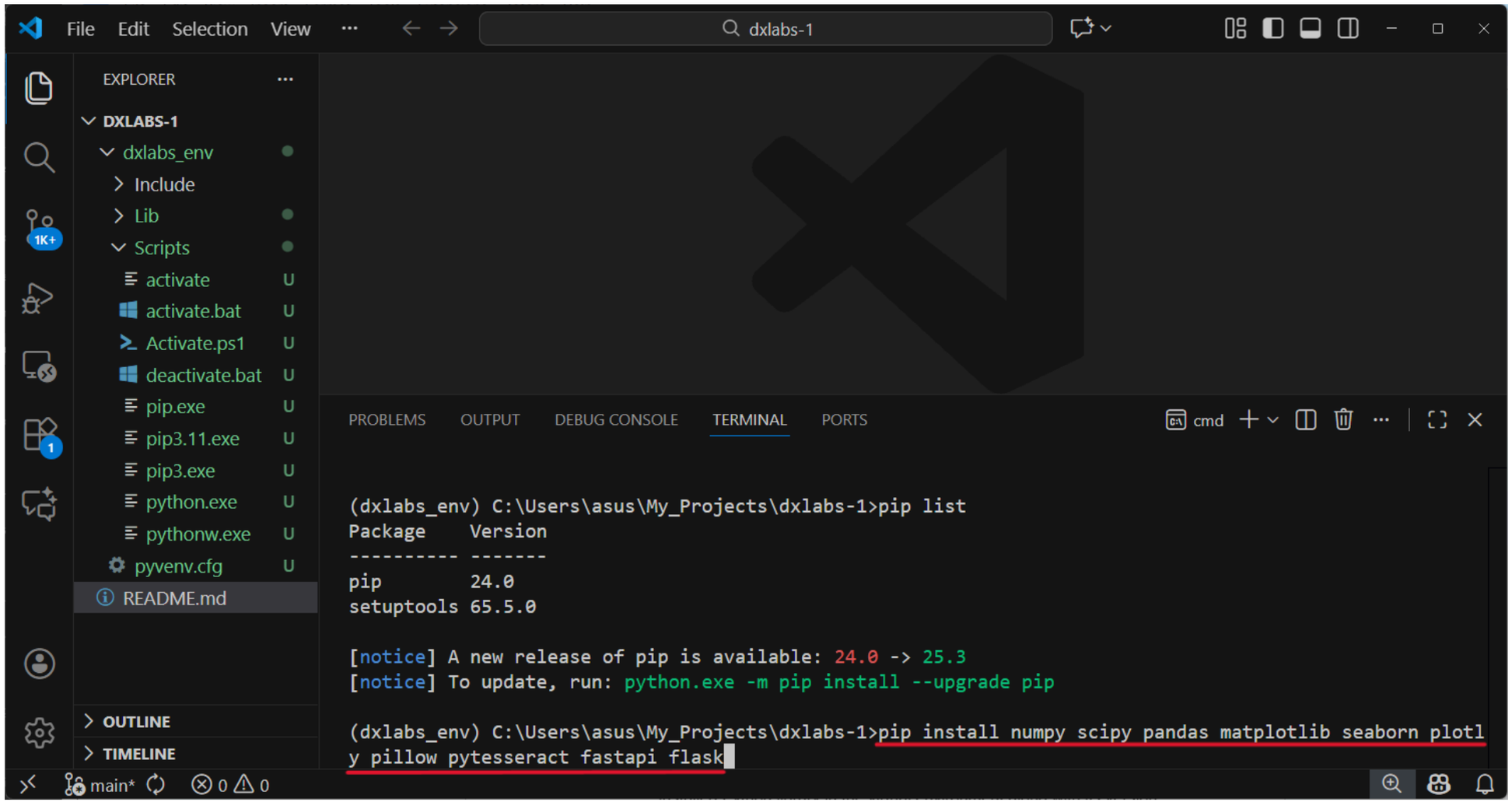


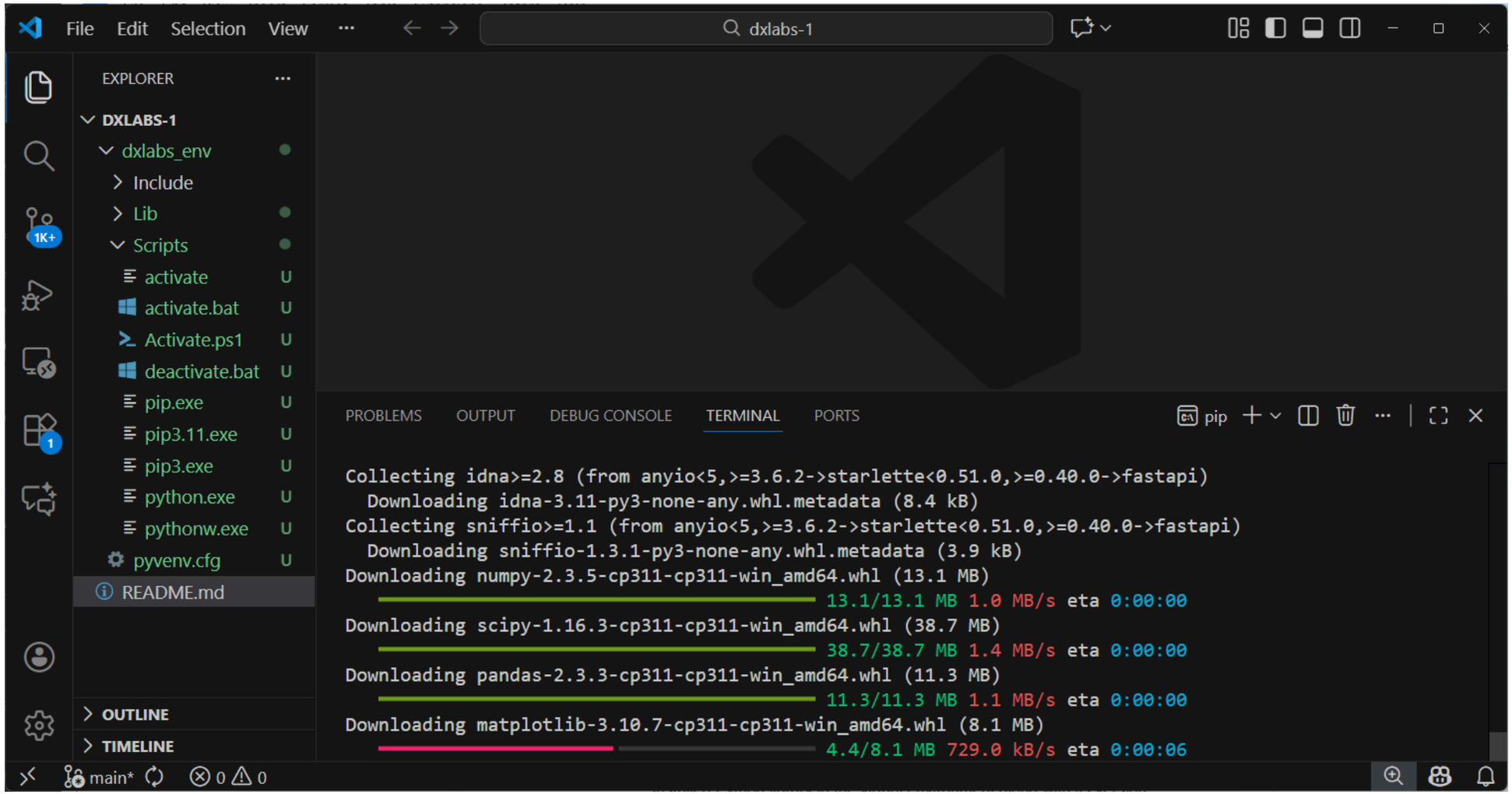

File
Edit
Selection
View
dxlabs-1
EXPLORER
DXLABS-1
dxlabs_env
Include
Lib
Scripts
activate
activate.bat
Activate.ps1
deactivate.bat
pip.exe
pip3.11.exe
pip3.exe
python.exe
pythonw.exe
pyvenv.cfg
README.md
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
cmd
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.
C:\Users\asus\My_Projects\dxlabs-1>python3 -m venv dxlabs_env
C:\Users\asus\My_Projects\dxlabs-1>dxlabs_env\Scripts\activate
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>
main*
0
0

b. Linux/macOS: `source myenv/bin/activate`
c. Note: Make sure to replace `myenv` with the name of your virtual environment.

Installing Python libraries within the environment

1. Now that you have created a Python virtual environment, you need to install the Python libraries that you will need in your Python project. To do so, use the following command in your command line interface in VS Code:
   a. `pip install package_name` (replace `package_name` with the name of the Python library that you would like to install)
2. For this training program, we will need to install several Python libraries for the activities. Use the following command in your command line interface in VS Code to install the Python libraries needed for this training.
   a. `pip install numpy scipy pandas matplotlib seaborn plotly pillow pytesseract fastapi flask`

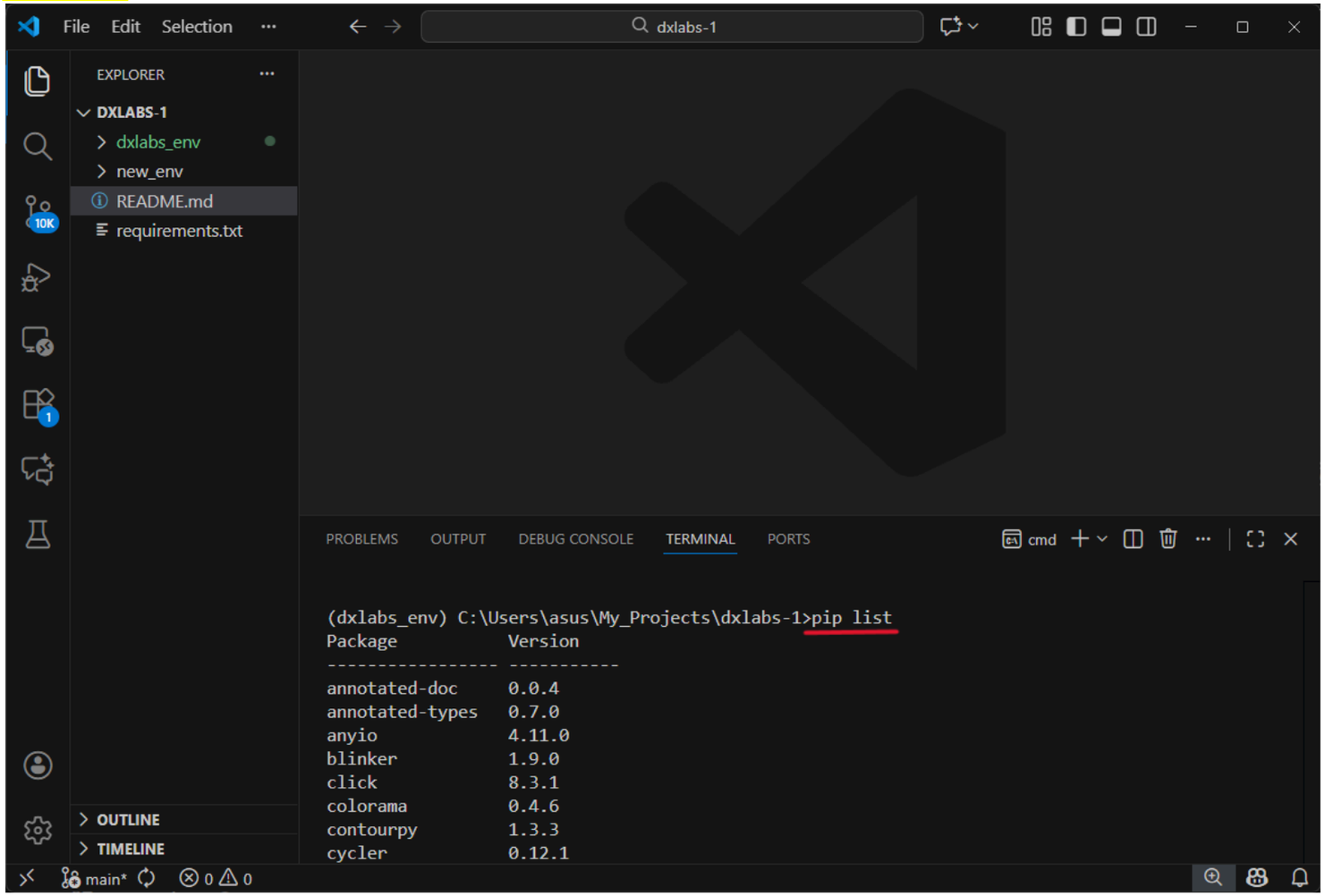

File Edit Selection View
dxlabs-1
EXPLORER
DXLABS-1
dxlabs_env
Include
Lib
Scripts
activate
activate.bat
Activate.ps1
deactivate.bat
pip.exe
pip3.11.exe
pip3.exe
python.exe
pythonw.exe
pyvenv.cfg
README.md
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
cmd
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>pip list
Package Version
---------- -------
pip 24.0
setuptools 65.5.0
[notice] A new release of pip is available: 24.0 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>pip install numpy scipy pandas matplotlib seaborn plotly pillow pytesseract fastapi flask
main*
0 0



b. Note: Make sure that your Python virtual environment is activated before installing the Python libraries using `pip`.

Listing all installed Python libraries within the environment

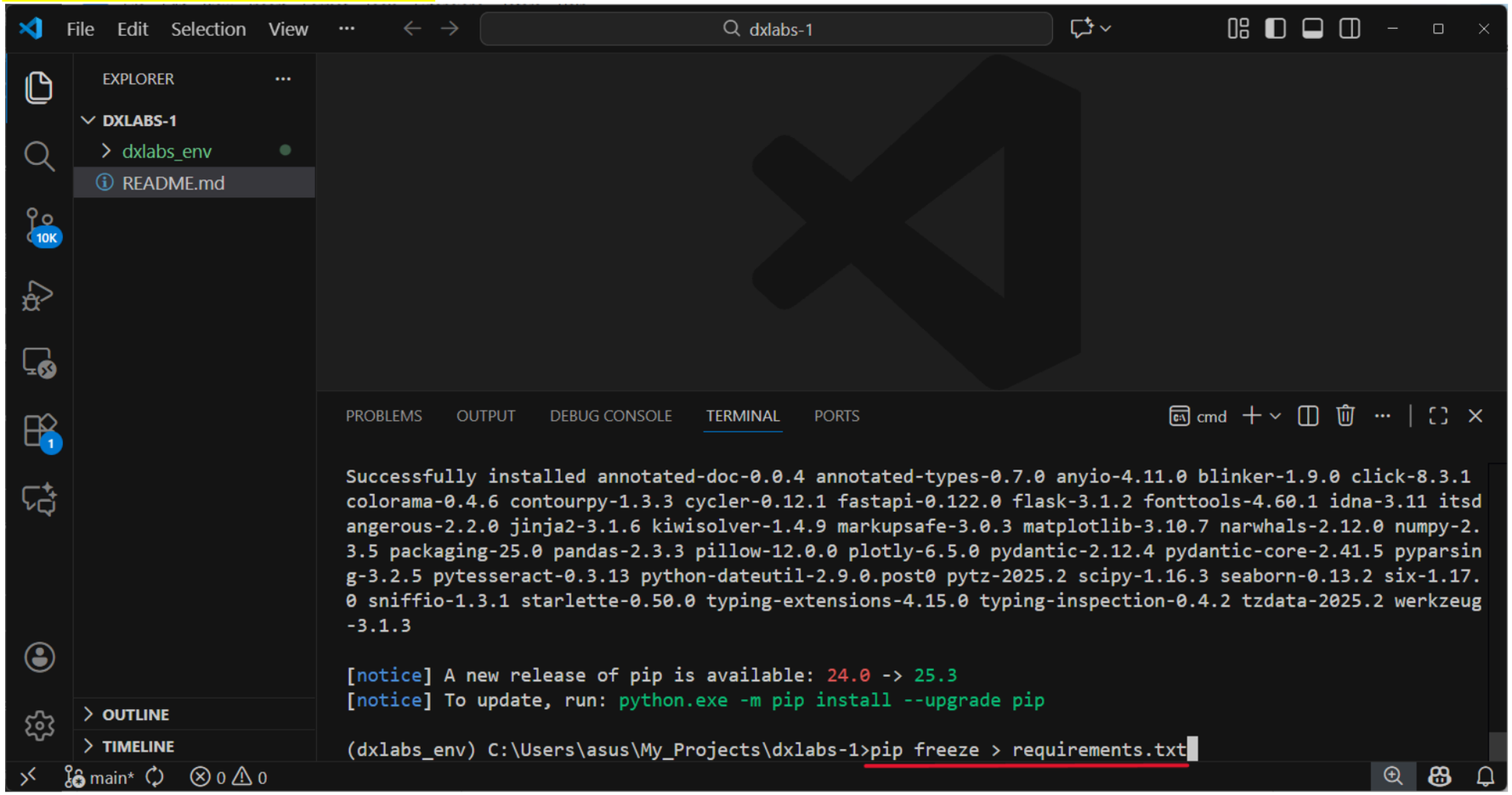
1. Once you have installed the necessary Python libraries in your virtual environment, you can list all these libraries and their versions using the following command:
    a. `pip list`

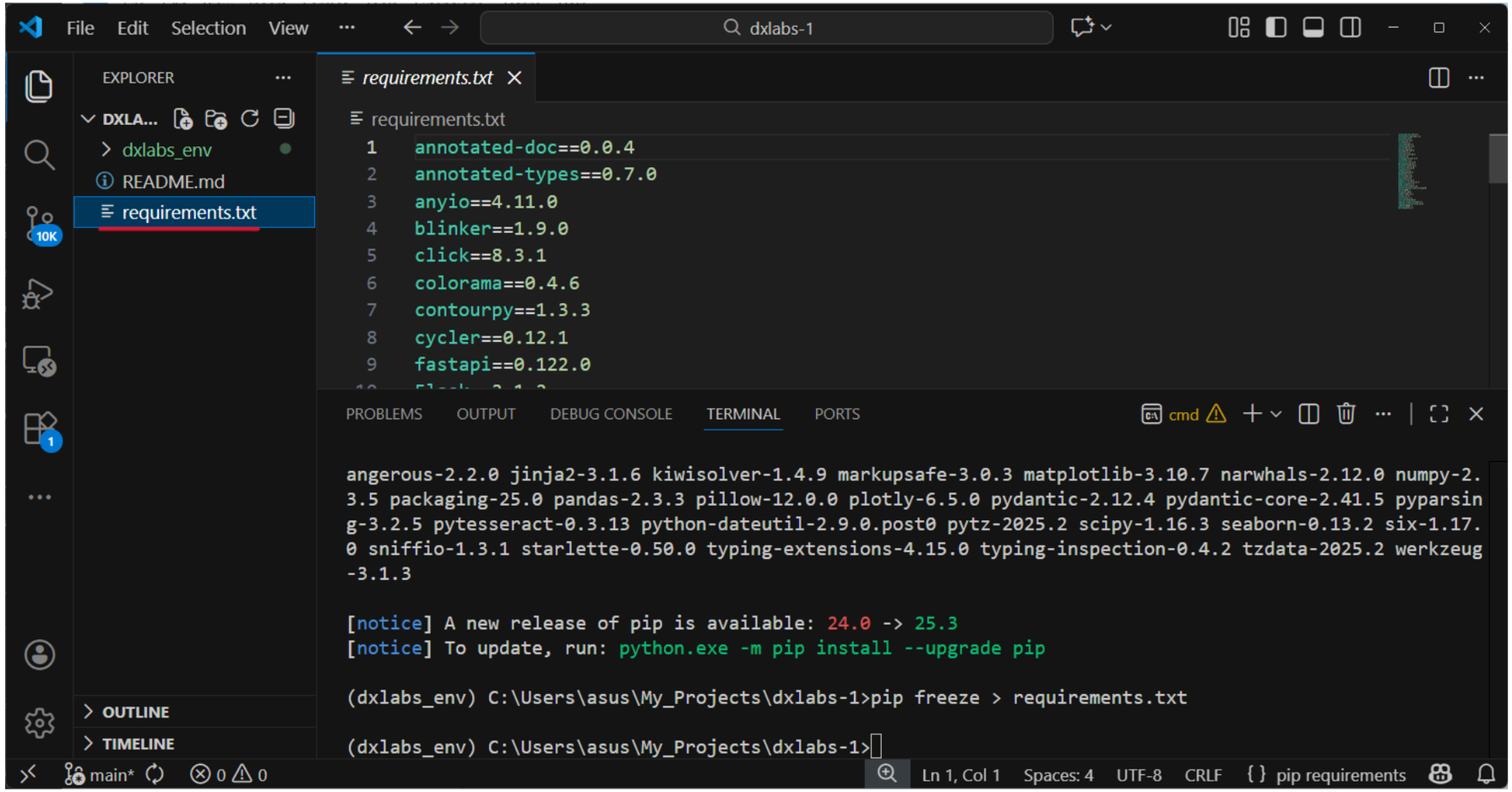


b. Note: Make sure that your Python virtual environment is activated before listing all the libraries installed within the virtual environment.

Exporting all installed Python libraries into a configuration file

1. You can export all the Python libraries installed in your virtual environment into a configuration text file so your virtual environment can be recreated in a separate environment on the same or a different computer. To do so, use the following command in your command line interface in VS Code:
   a. `pip freeze > requirements.txt`

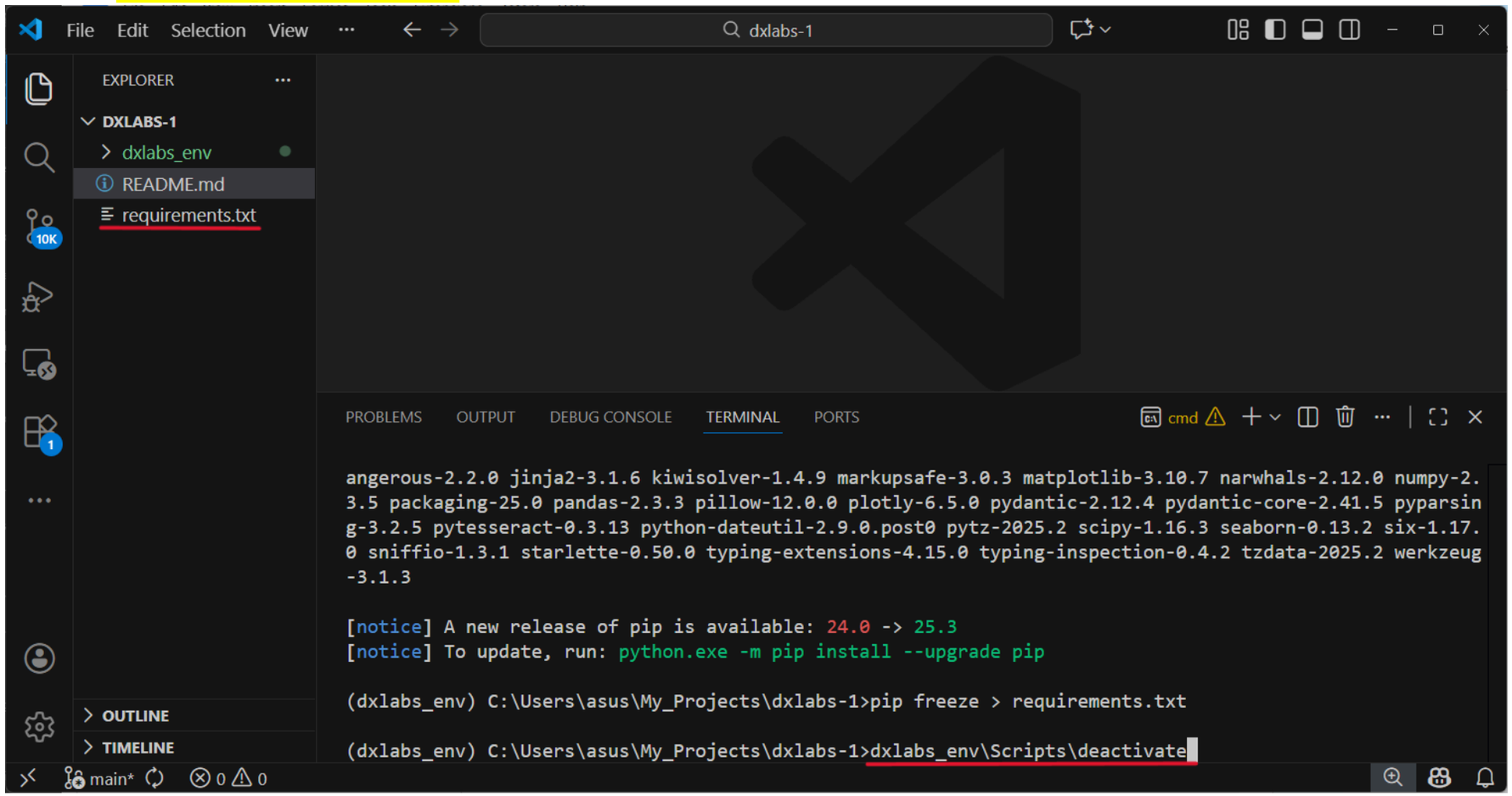


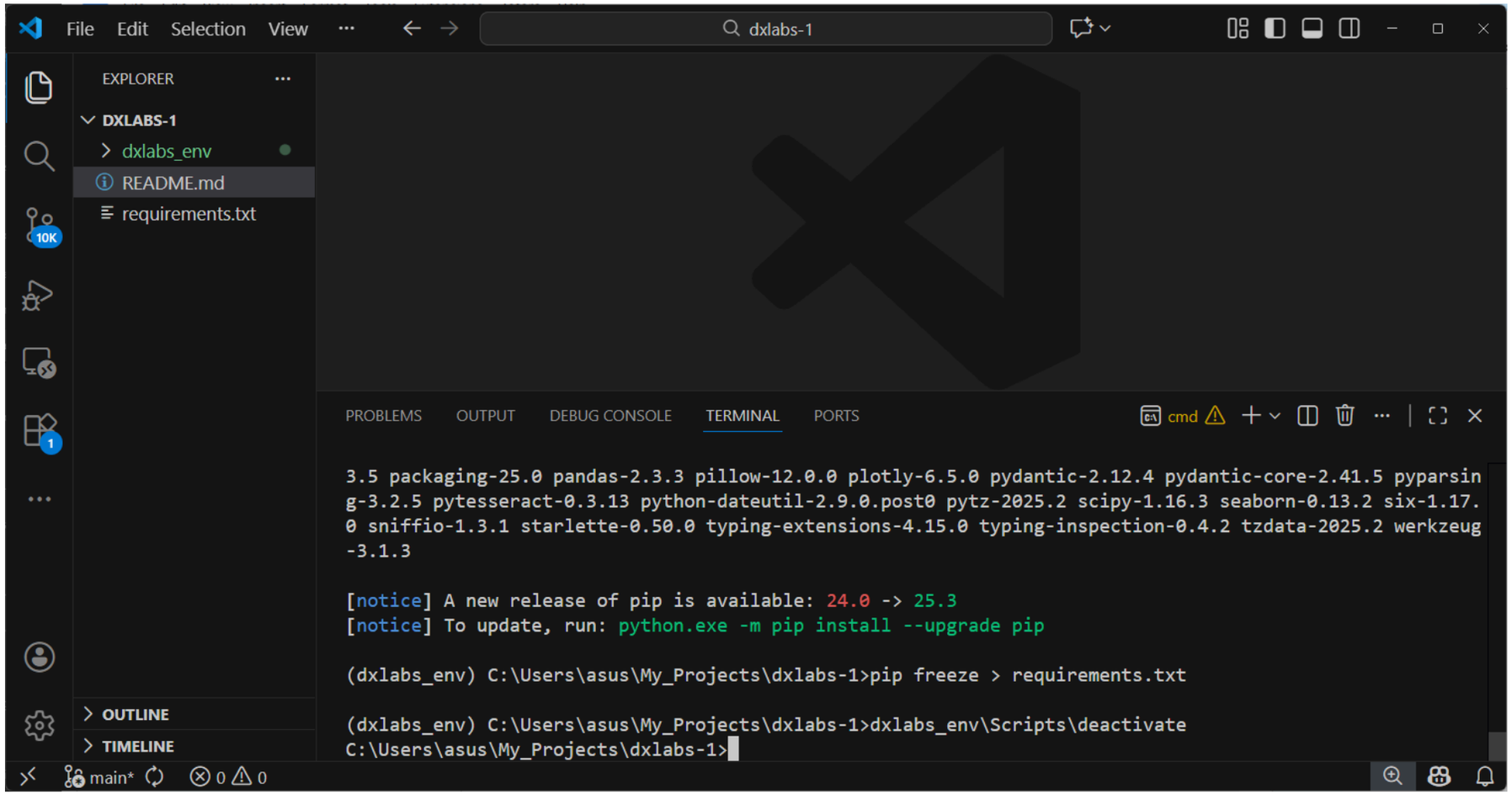


b. This command will create a `requirements.txt` file listing every installed Python library in the virtual environment along with its version.
c. Note: Make sure that your Python virtual environment is activated before exporting the Python libraries to a configuration file `requirements.txt`.

Recreating the same Python virtual environment

1. Given the generated `requirements.txt file`, you can recreate your Python virtual environment in your computer or in another computer with an installed `Python 3.3` or later. To do so, use the following sequence of commands in the command line interface in your VS Code.
    a. Deactivate your current Python virtual environment using the following command prompt.
        i. Windows: `menv\Scripts\deactivate`

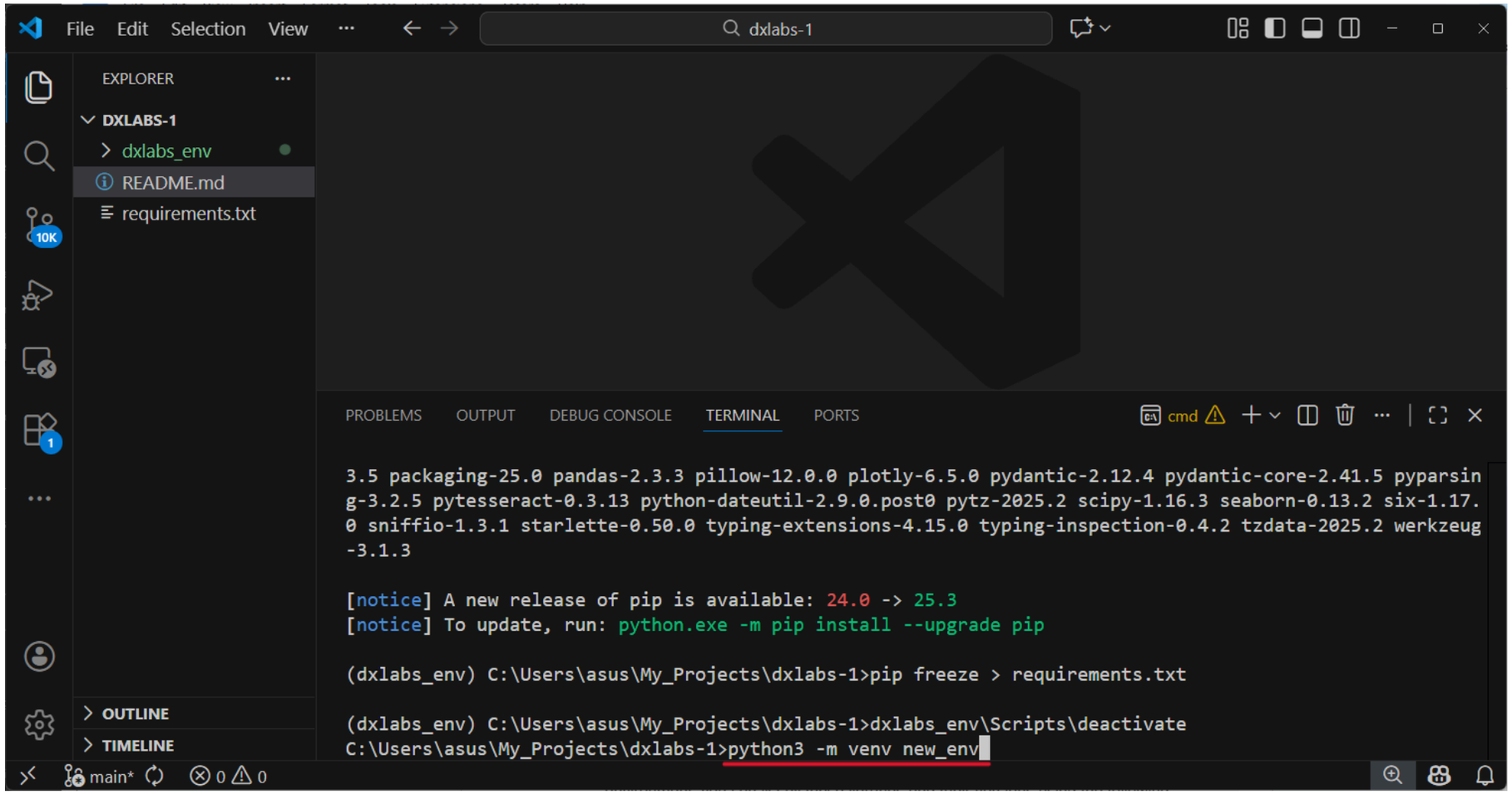




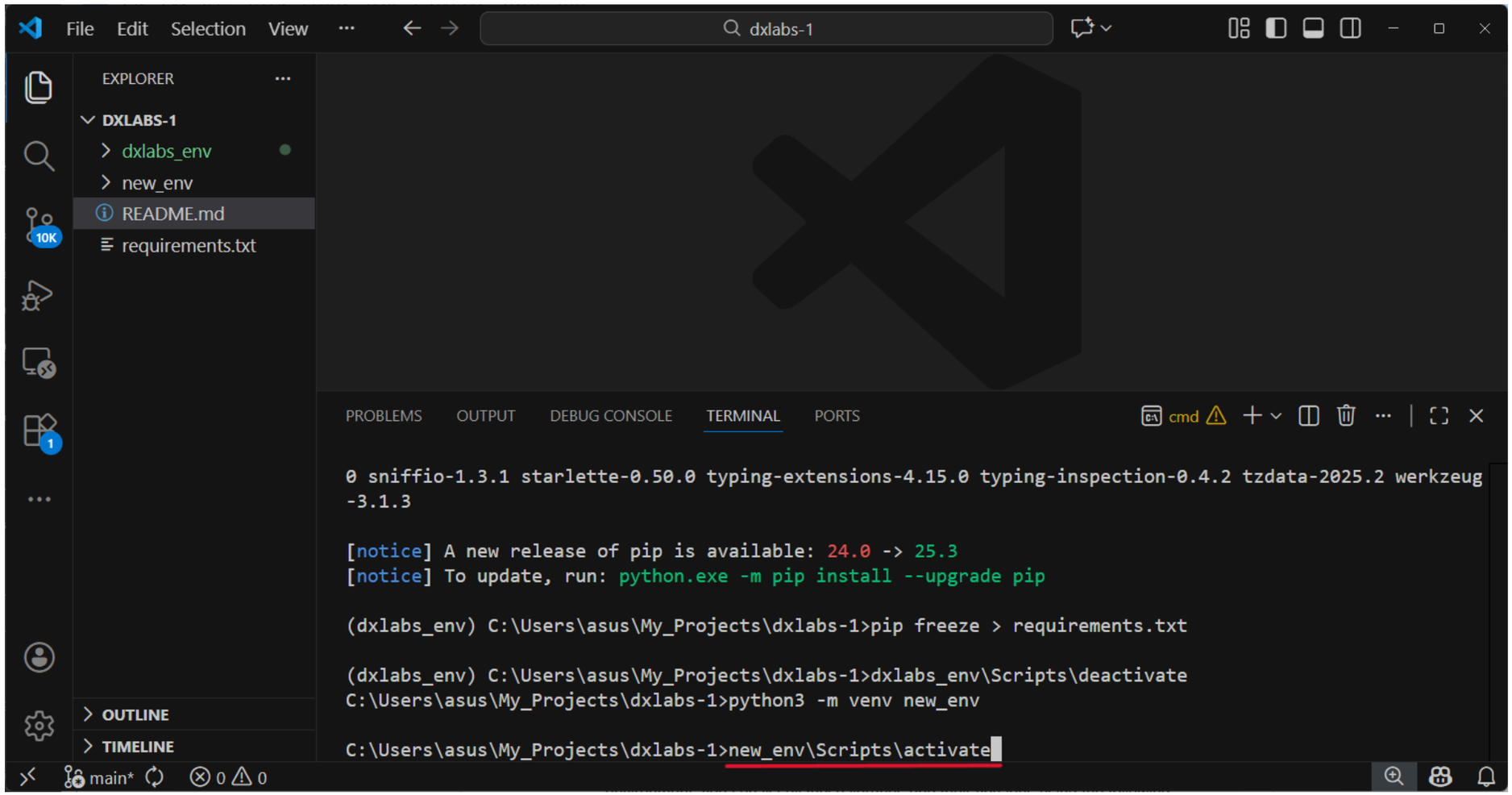
ii. macOS/Linux: `deactivate`

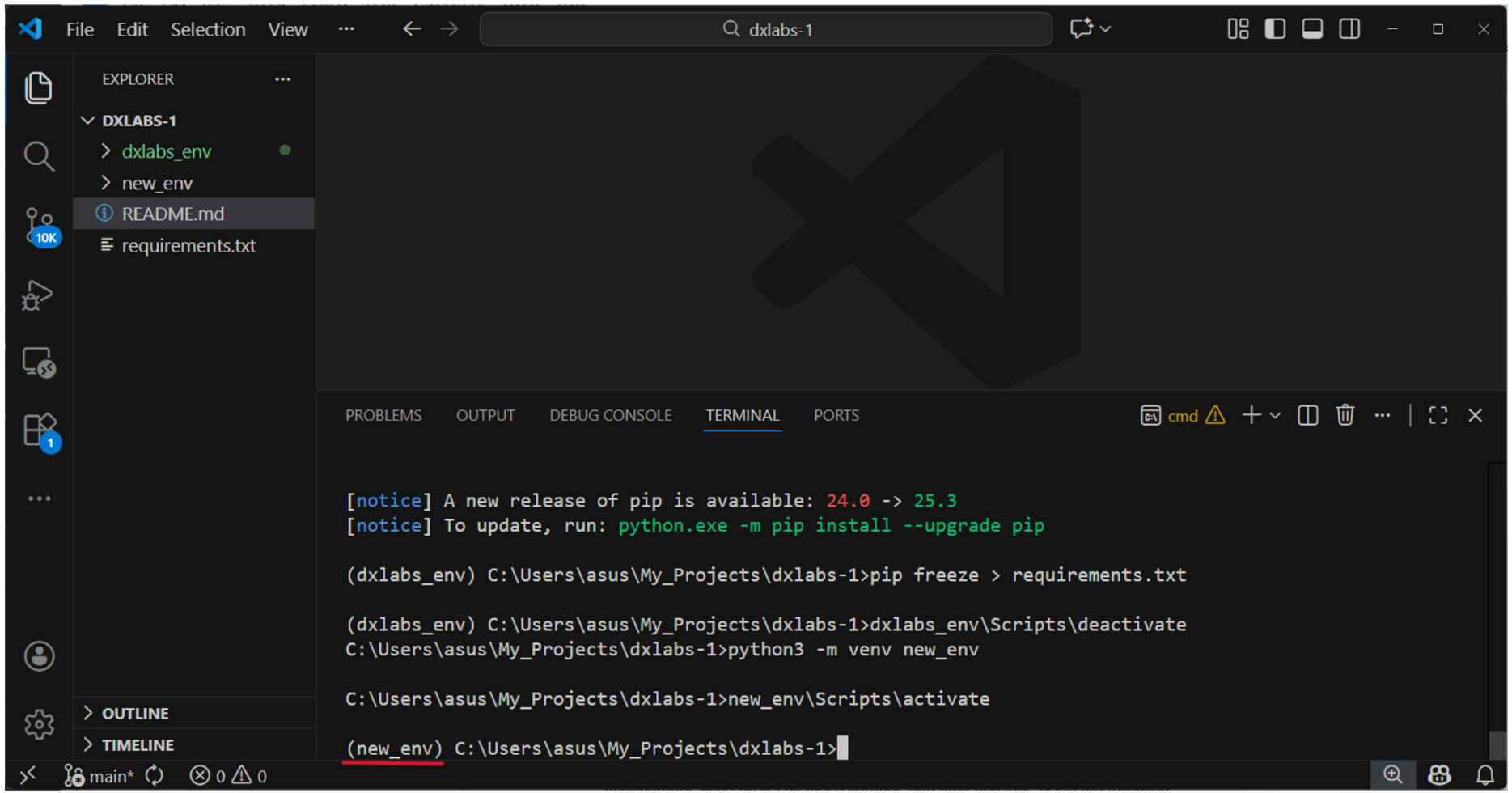
b. Create a new Python virtual environment on which you will install the Python libraries listed in the `requirements.txt` file. To do so, use the following command prompt in the command line interface in VS Code. Note that you can use any name for your new Python virtual environment.

i. `python3 -m venv new_env`



c. Activate the newly created Python virtual environment using the following command prompt:
   i. Windows: `new_env\Scripts\activate`

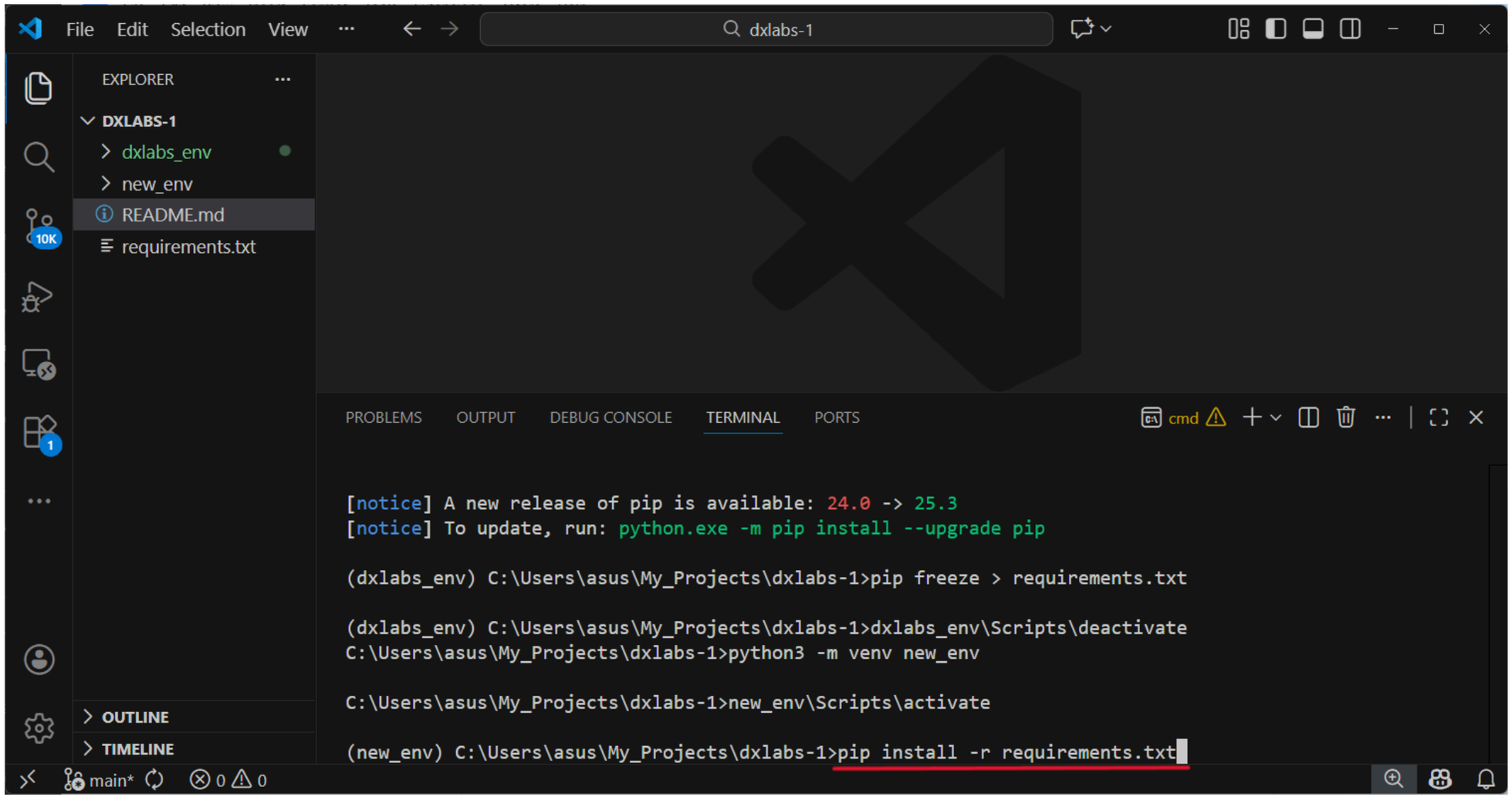

File Edit Selection View
dxlabs-1
EXPLORER
DXLABS-1
dxlabs_env
new_env
README.md
requirements.txt
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
cmd
0 sniffio-1.3.1 starlette-0.50.0 typing-extensions-4.15.0 typing-inspection-0.4.2 tzdata-2025.2 werkzeug
-3.1.3
[notice] A new release of pip is available: 24.0 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>pip freeze > requirements.txt
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>dxlabs_env\Scripts\deactivate
C:\Users\asus\My_Projects\dxlabs-1>python3 -m venv new_env
C:\Users\asus\My_Projects\dxlabs-1>new_env\Scripts\activate
main*
0 0

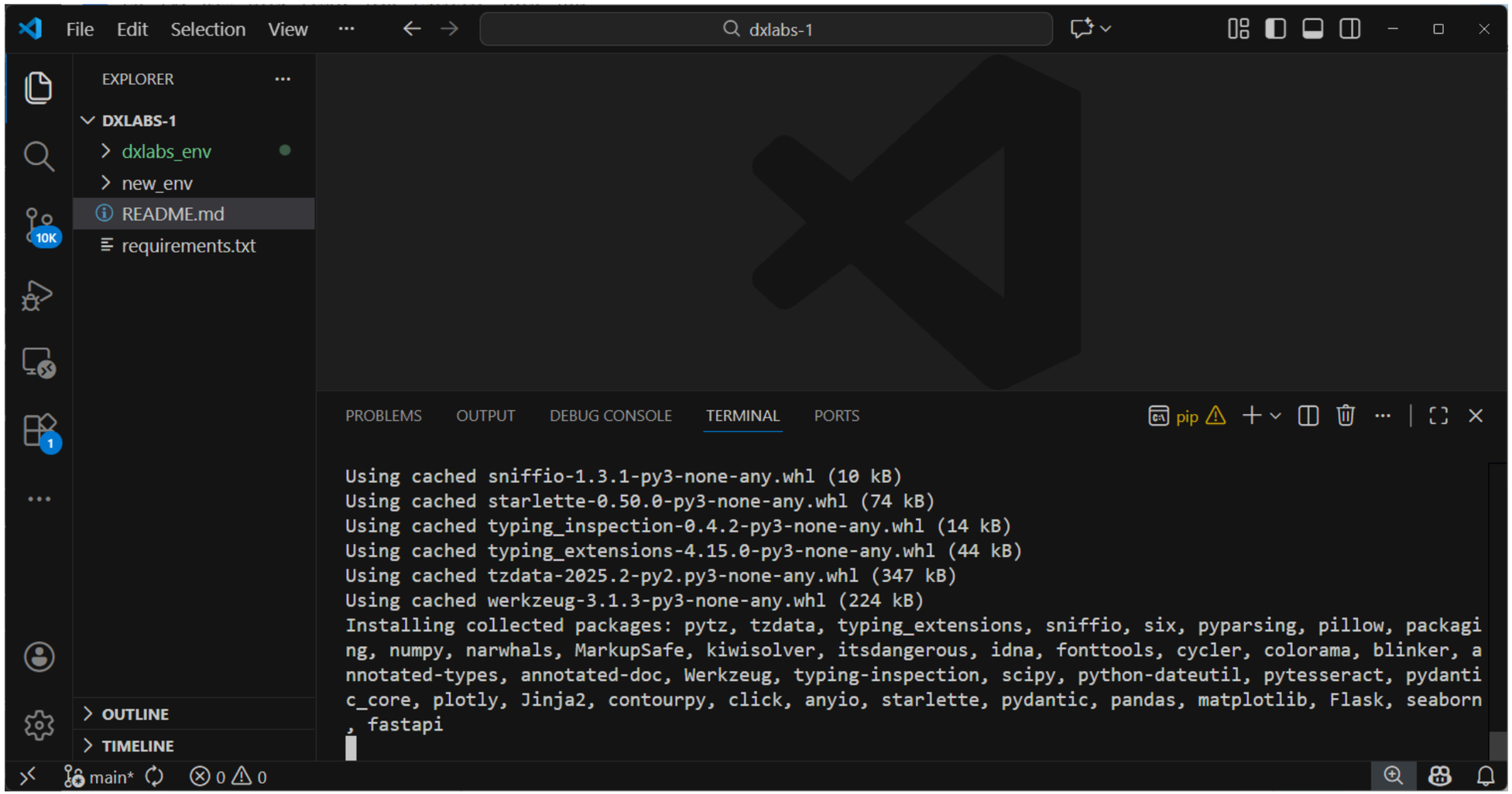


ii. macOS/Linux: `source new_env/bin/activate`

d. Install the necessary Python libraries within the activate new Python virtual environment using the following command prompt:

i. `pip install -r requirements.txt`


File
Edit
Selection
View
dxlabs-1
EXPLORER
DXLABS-1
dxlabs_env
new_env
README.md
requirements.txt
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
cmd
[notice] A new release of pip is available: 24.0 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>pip freeze > requirements.txt
(dxlabs_env) C:\Users\asus\My_Projects\dxlabs-1>dxlabs_env\Scripts\deactivate
C:\Users\asus\My_Projects\dxlabs-1>python3 -m venv new_env
C:\Users\asus\My_Projects\dxlabs-1>new_env\Scripts\activate
(new_env) C:\Users\asus\My_Projects\dxlabs-1>pip install -r requirements.txt
main*
0 0



e. Note: Make sure that you have created a new virtual environment and it is active before running the command.

Now that you have learned how to create a virtual environment in Python and how to install Python libraries within the context of this virtual environment, you can now access these libraries within your Python code. You can simply install new Python libraries within an active virtual environment as you encounter the need in your Python programming.

GitHub x VS Code

Guide on How to Use GitHub with Visual Studio Code for Code Versioning

Version 1.0

The goal of this document is to instruct you on how to keep versions of your source code as you develop it through time. Keeping versions of your code will allow you to go back to any version as you find it necessary in your code development journey.

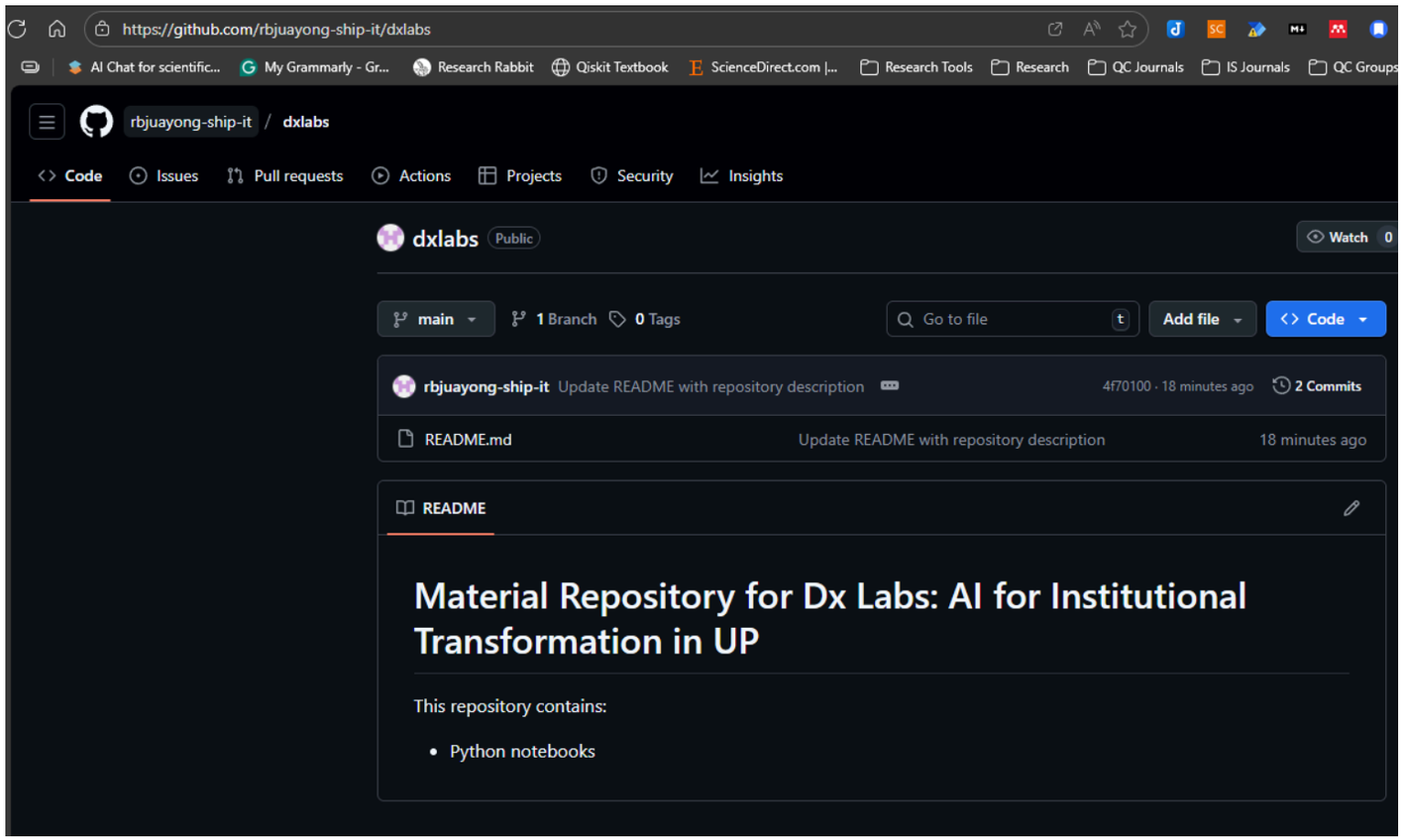
To achieve this goal, we have the following objectives:

1. To instruct you on how to copy (fork) a repository from someone's GitHub account into your own GitHub account,
2. To instruct you on how to link (clone) your GitHub repository into your Visual Studio Code editor so you can make changes to your code locally on your computer.
3. To instruct you on how to add local changes to a file in your cloned repository.
4. To instruct you on how to reflect the local changes that you made to a file in your cloned code repository to your remote GitHub repository.
5. To instruct you on how to view the changes that you have made and reflected on your remote GitHub repository's web page.

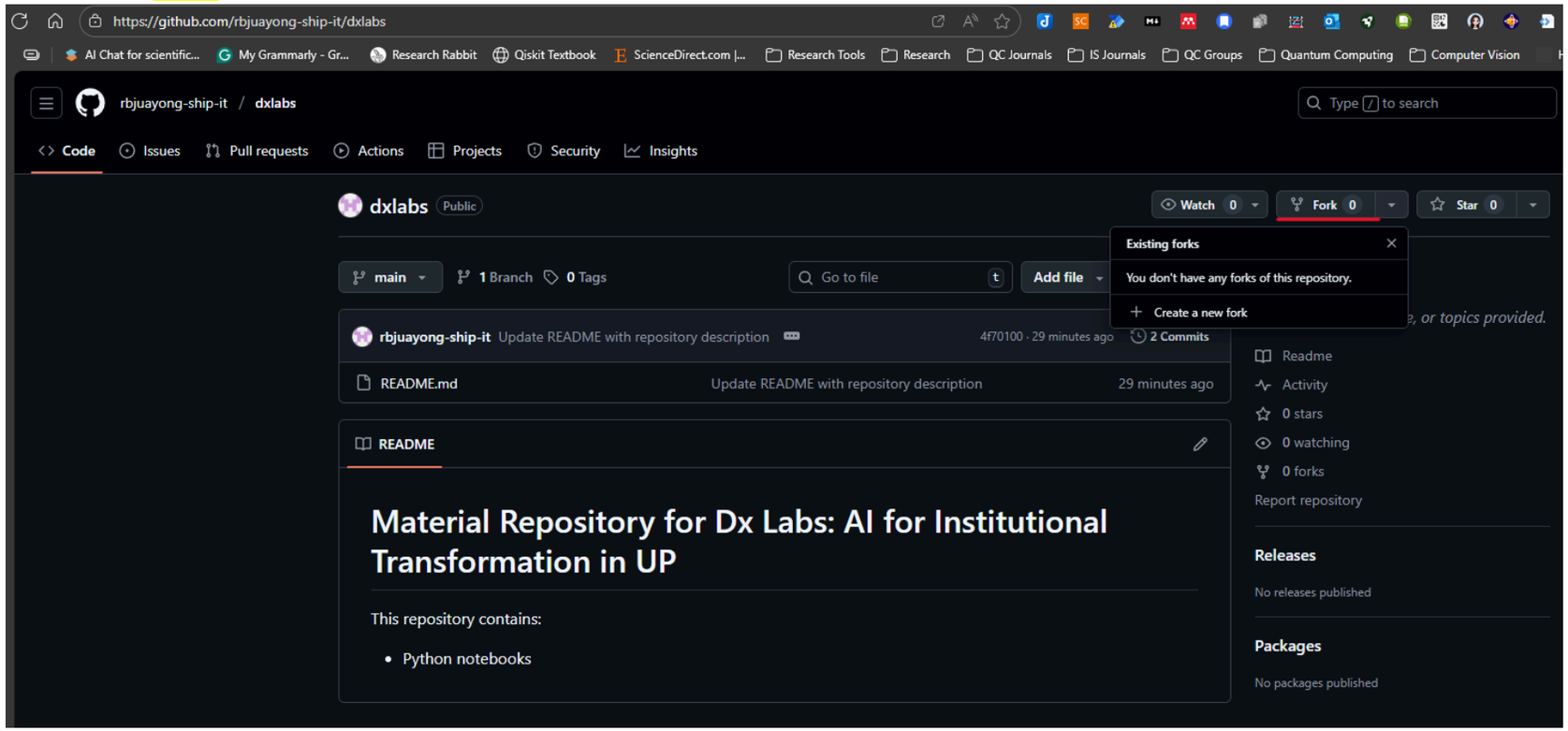
Table of Contents

1. Forking a Repository from Someone's GitHub Account
2. Cloning a Forked Repository and Opening it in Your Visual Studio Code Editor
3. Adding Local Changes to a File in the Cloned Code Repository
4. Reflecting Local Changes in a File to your Remote GitHub Repository
5. Viewing the Changes in your GitHub Repository

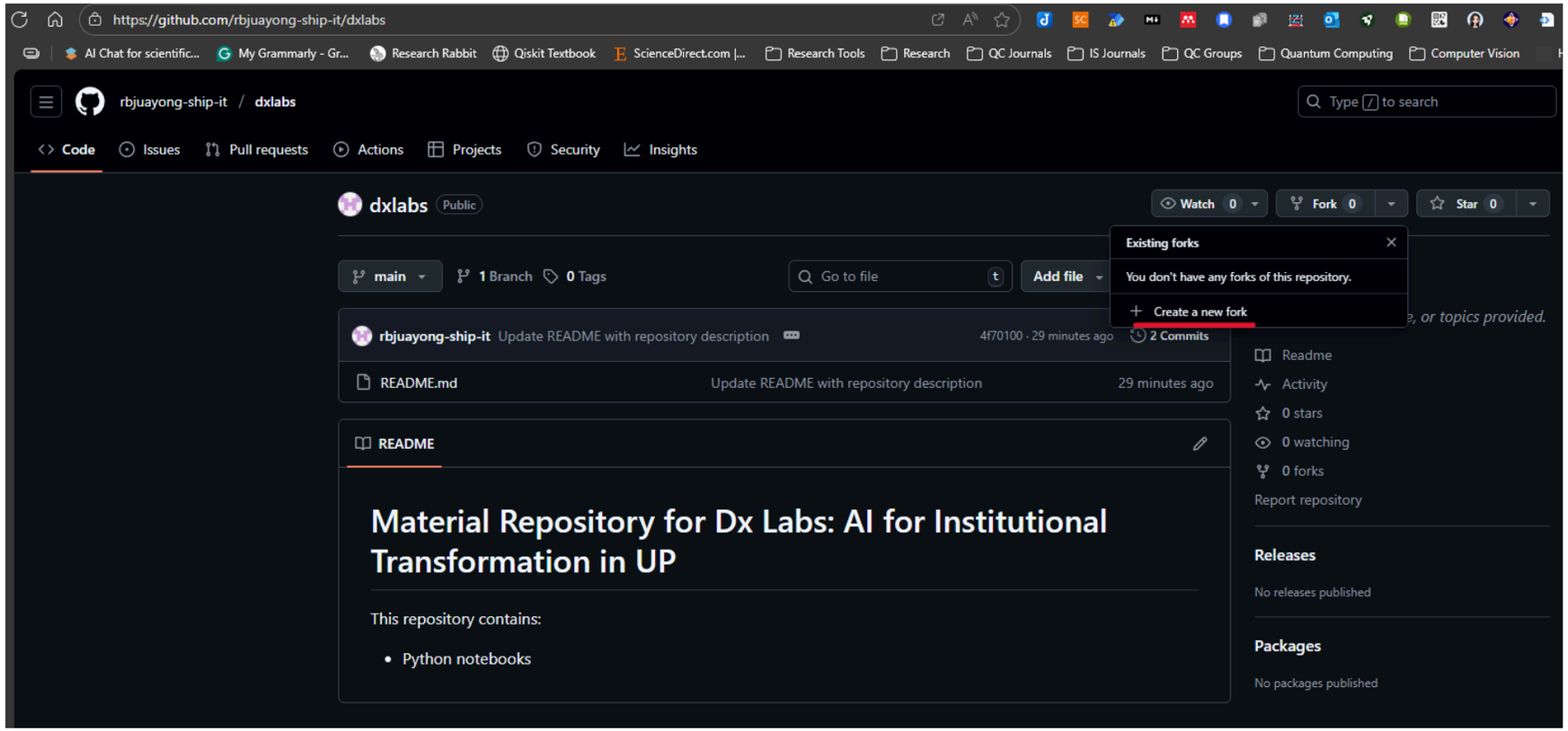
1. **Forking a Repository from Someone's GitHub Account**
    1. Open the GitHub page in your web browser.
    2. Log in to GitHub using your own GitHub user account.
    3. Open a new tab in your web browser and access the GitHub repository that you want to copy. In this case, we will need to copy the `dxlabs` code repository from Doc Rich's GitHub account at this address: https://github.com/rbjuayong-ship-it/dxlabs



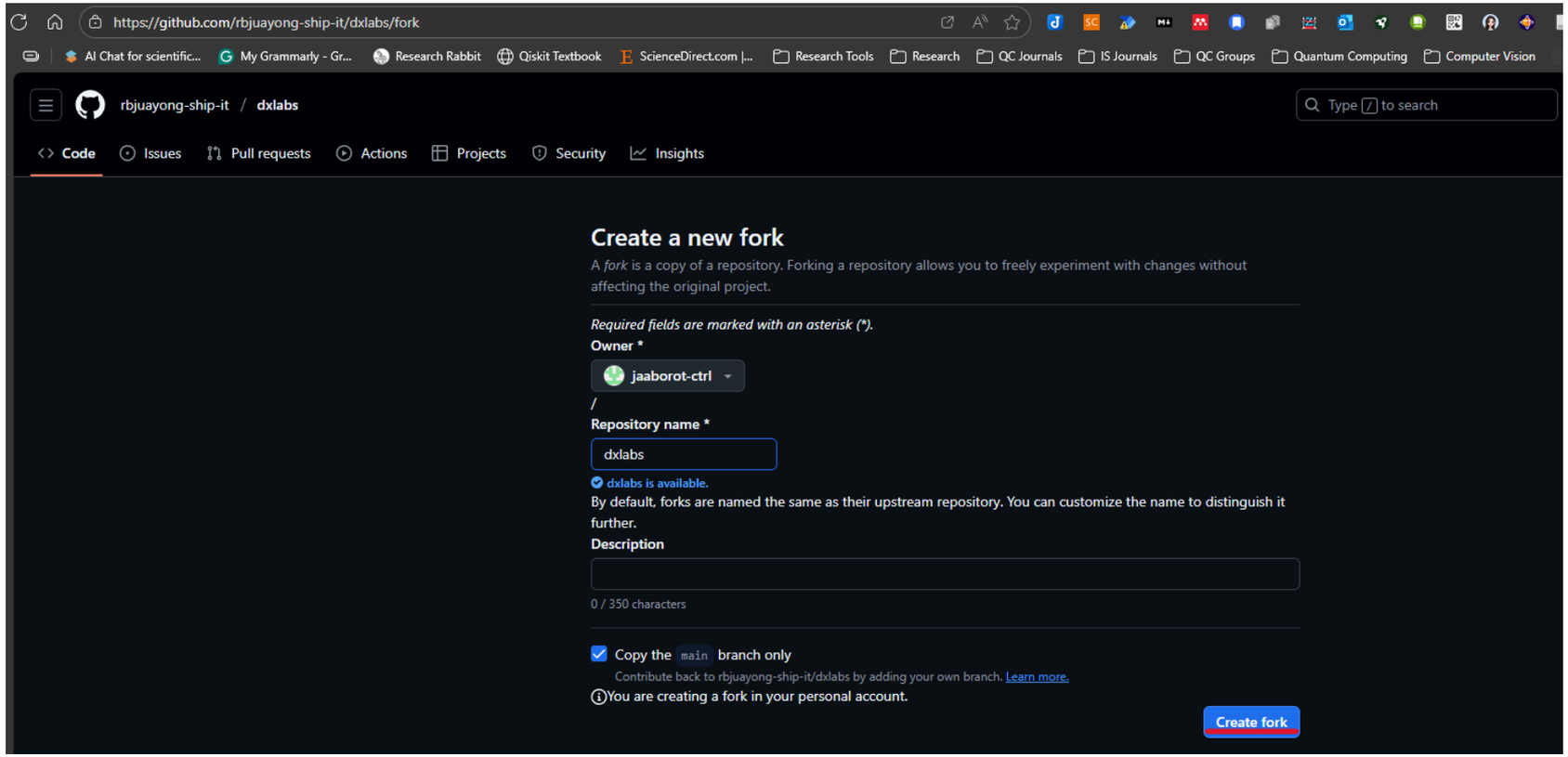
4. Fork the `dxlabs` code repository in Doc Rich's GitHub account into your own GitHub account.
    a. Click on the `Fork` button in Doc Rich's dxlabs GitHub repository, and a context menu will be displayed.



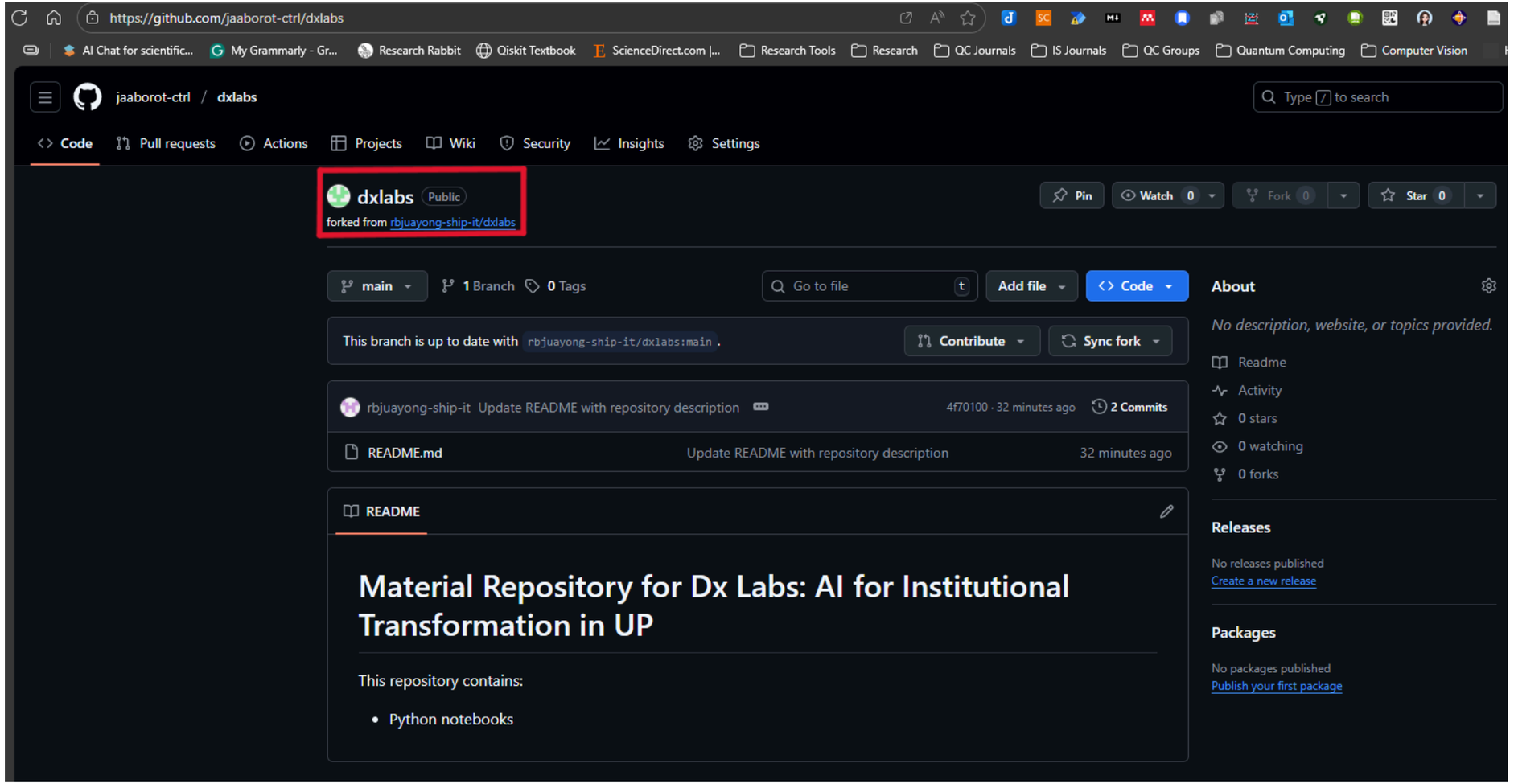
    b. Click on the `Create a new fork` button to fork the `dxlabs` repository from Doc Rich's GitHub account. You will be redirected to the page for forking Doc Rich's `dxlabs` repository.



5. Now that you are on the forking page of the `dxlabs` repository, keep the default entries and click on the `Create fork` button.

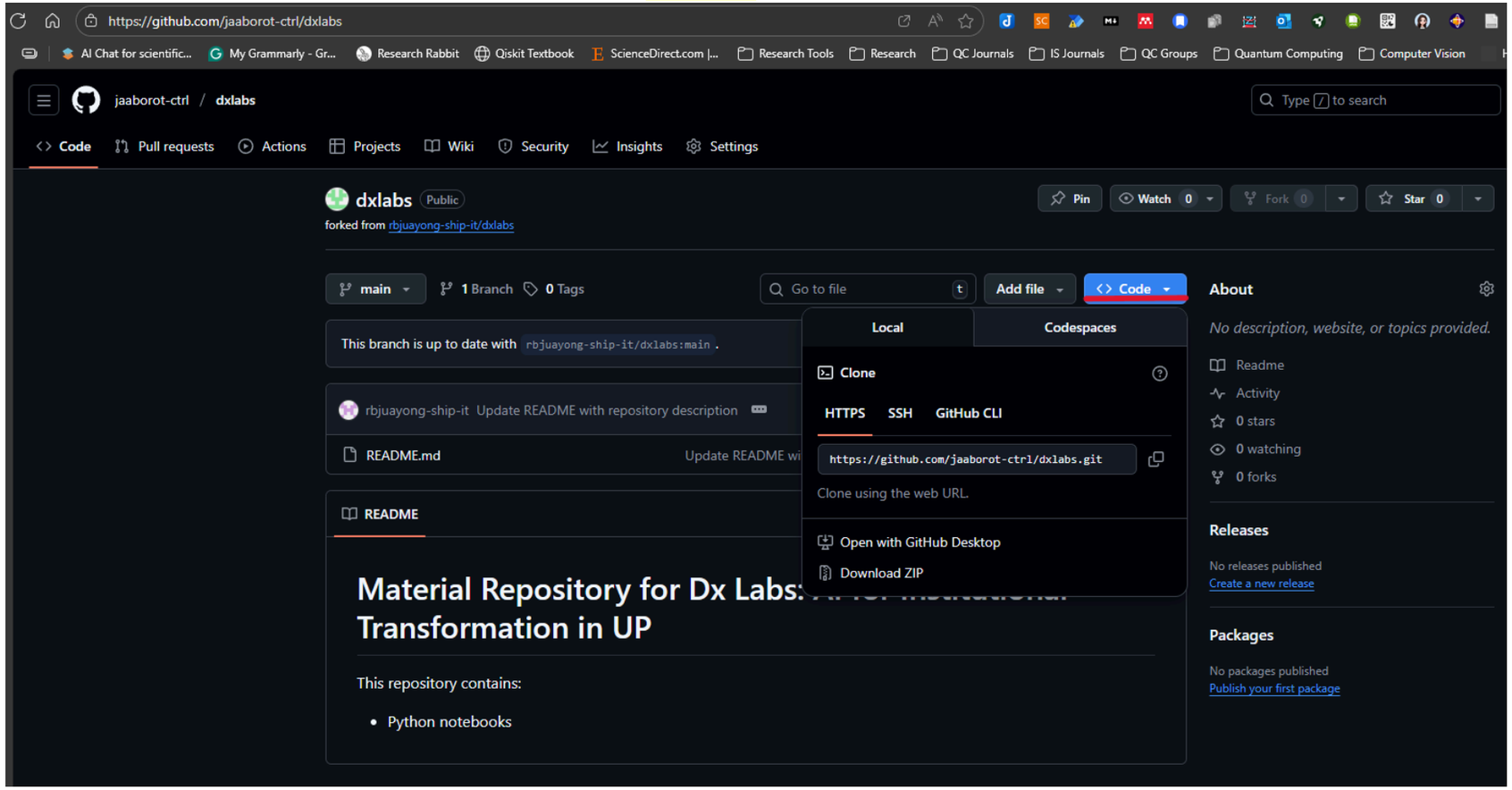

https://github.com/rbjuayong-ship-it/dxlabs/fork
AI Chat for scientific...
My Grammarly - Gr...
Research Rabbit
Qiskit Textbook
ScienceDirect.com |...
Research Tools
Research
QC Journals
IS Journals
QC Groups
Quantum Computing
Computer Vision
rbjuayong-ship-it / dxlabs
Type / to search
Code
Issues
Pull requests
Actions
Projects
Security
Insights
Create a new fork
A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.
Required fields are marked with an asterisk (*).
Owner *
jaaborot-ctrl
/
Repository name *
dxlabs
dxlabs is available.
By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.
Description
0 / 350 characters
Copy the main branch only
Contribute back to rbjuayong-ship-it/dxlabs by adding your own branch. Learn more.
You are creating a fork in your personal account.
Create fork

6. Verify that you have been redirected to your forked copy of the `dxlabs` repository on your GitHub page.

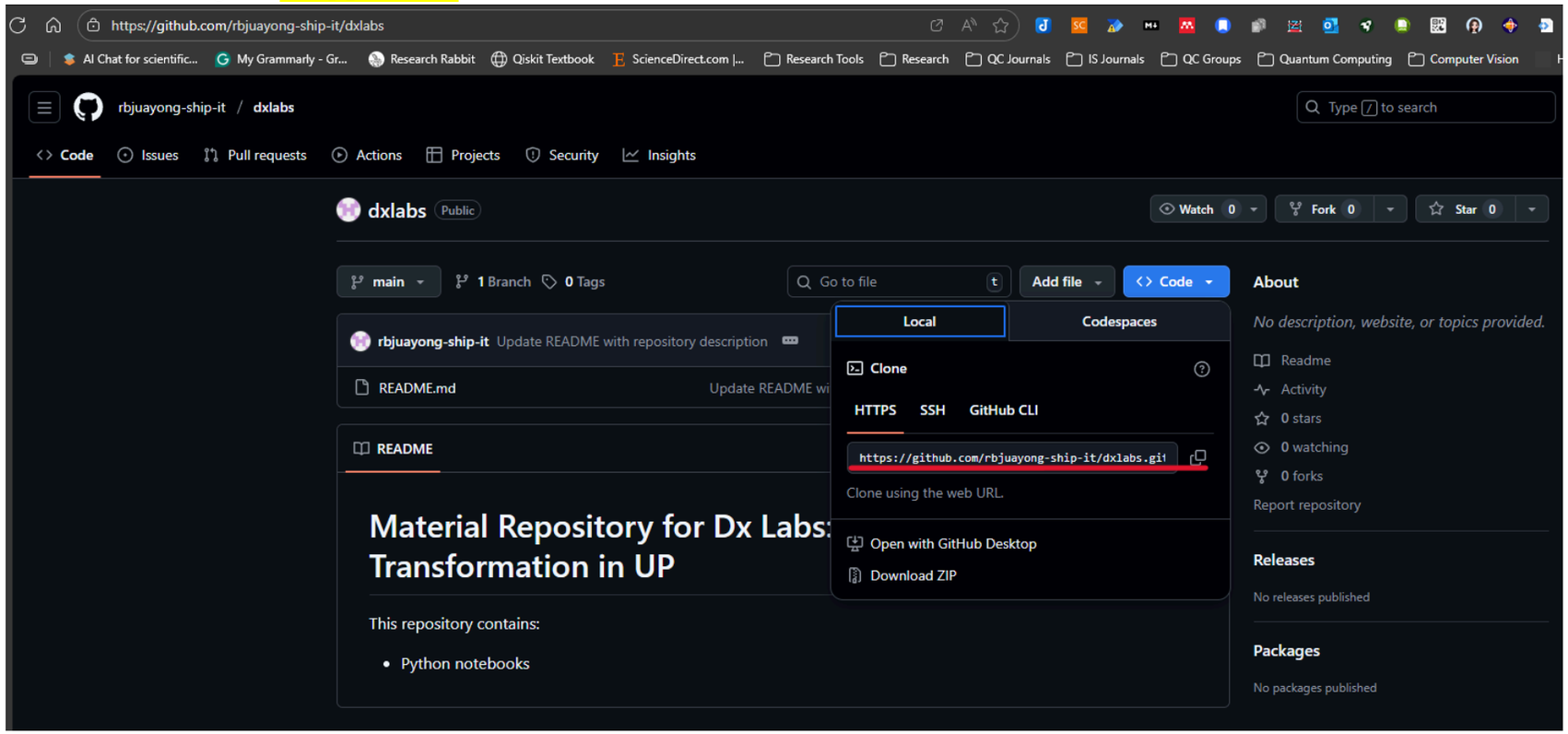


2. Cloning a Forked Repository and Opening it in Your Visual Studio Code Editor

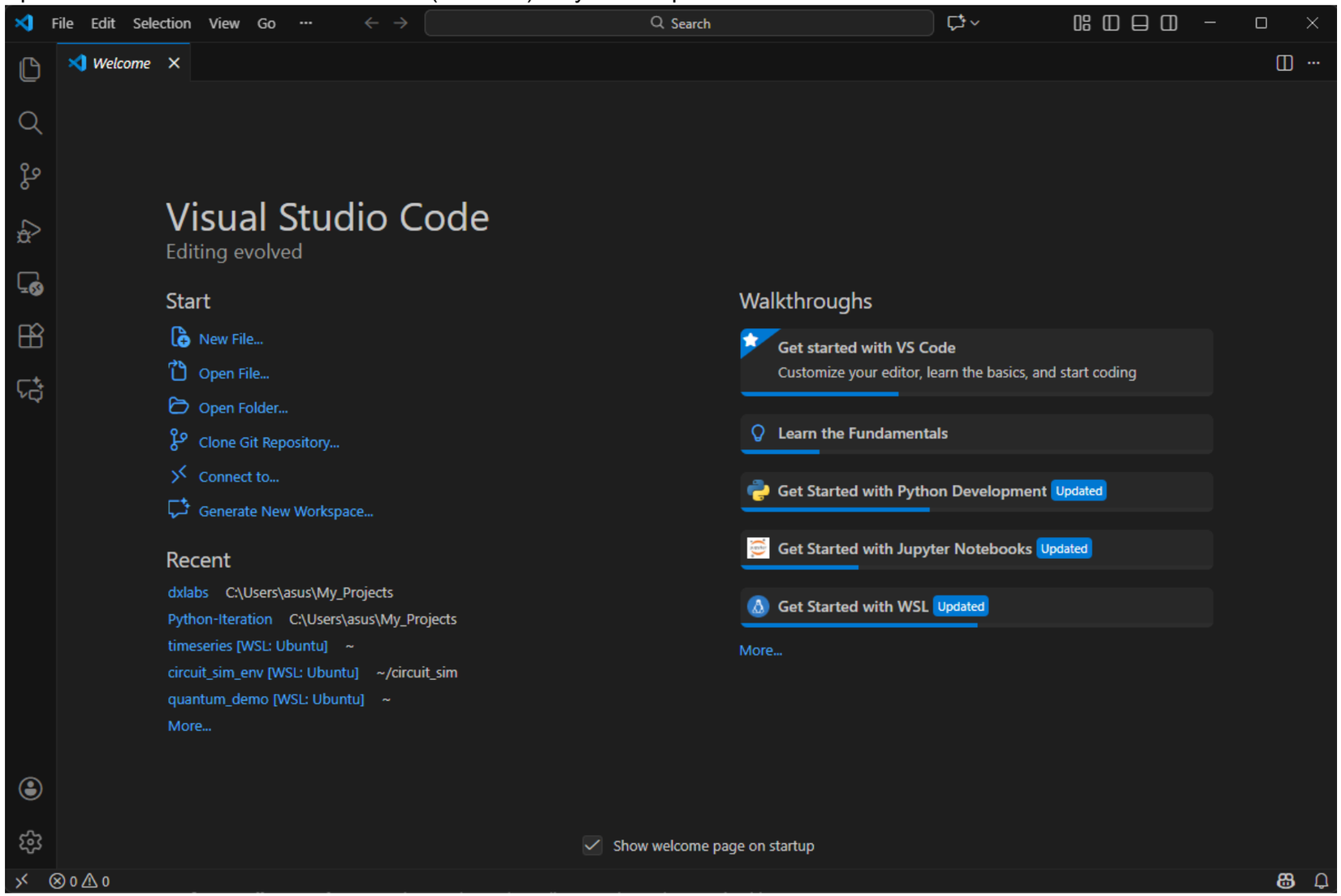
1. In the forked `dxlab` code repository page, click on the `<> Code` button.



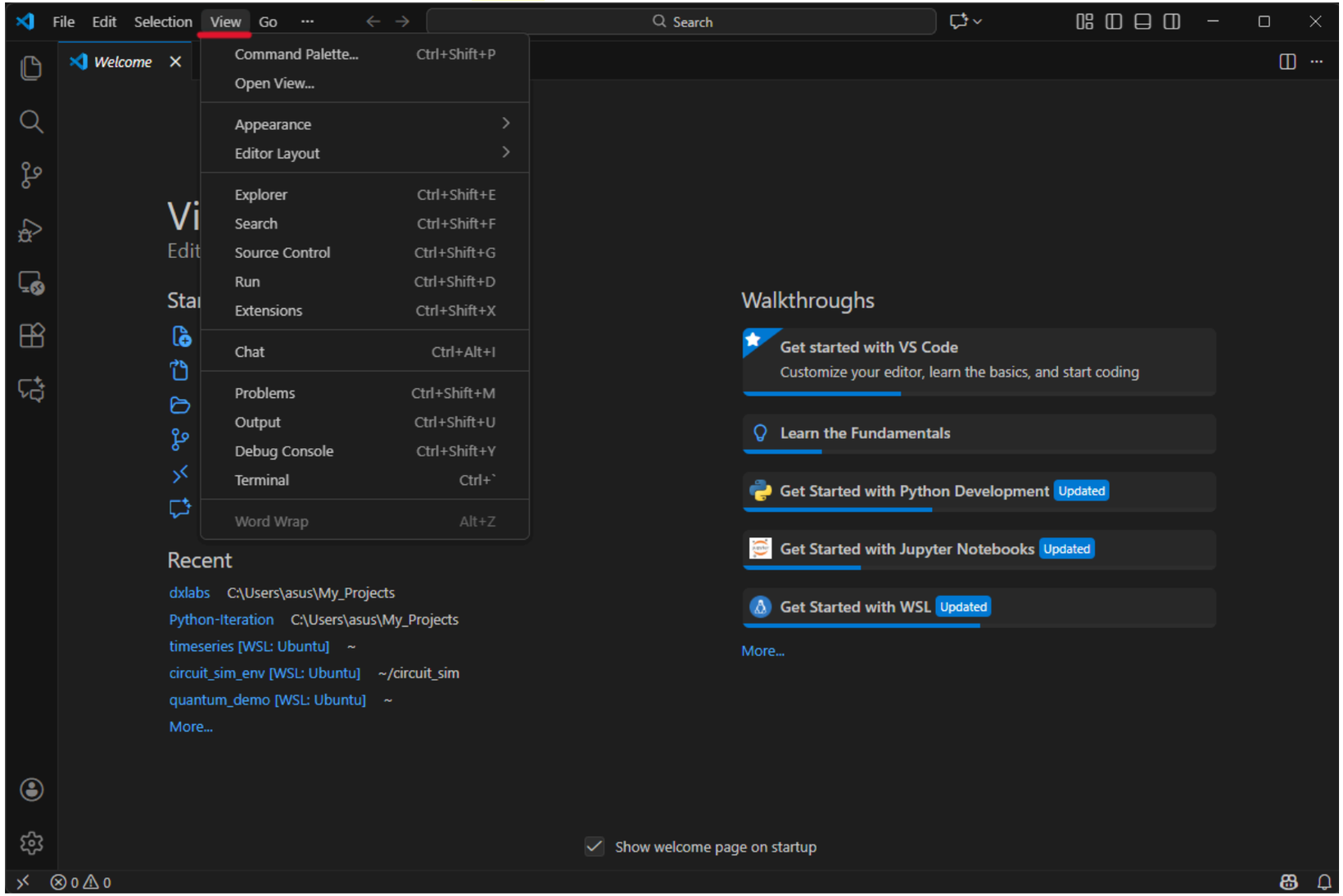
2. In the Local tab, under the HTTPS mini page, copy the link to the forked dxlab code repository by either directly selecting and copying the link in the link field or clicking the `"Copy link"` button.



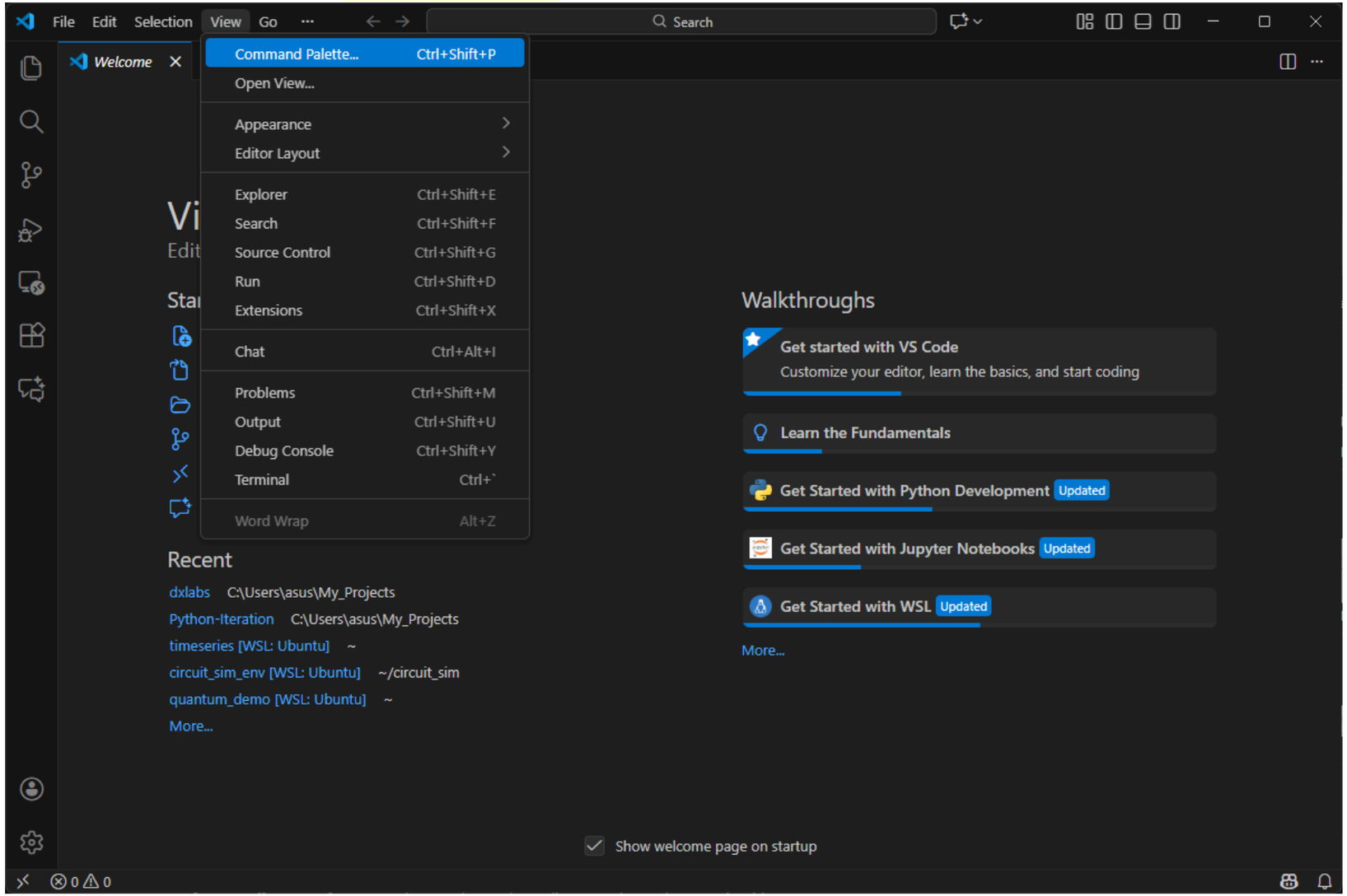
3. Open an instance of Visual Studio Code (VSCode) on your computer.



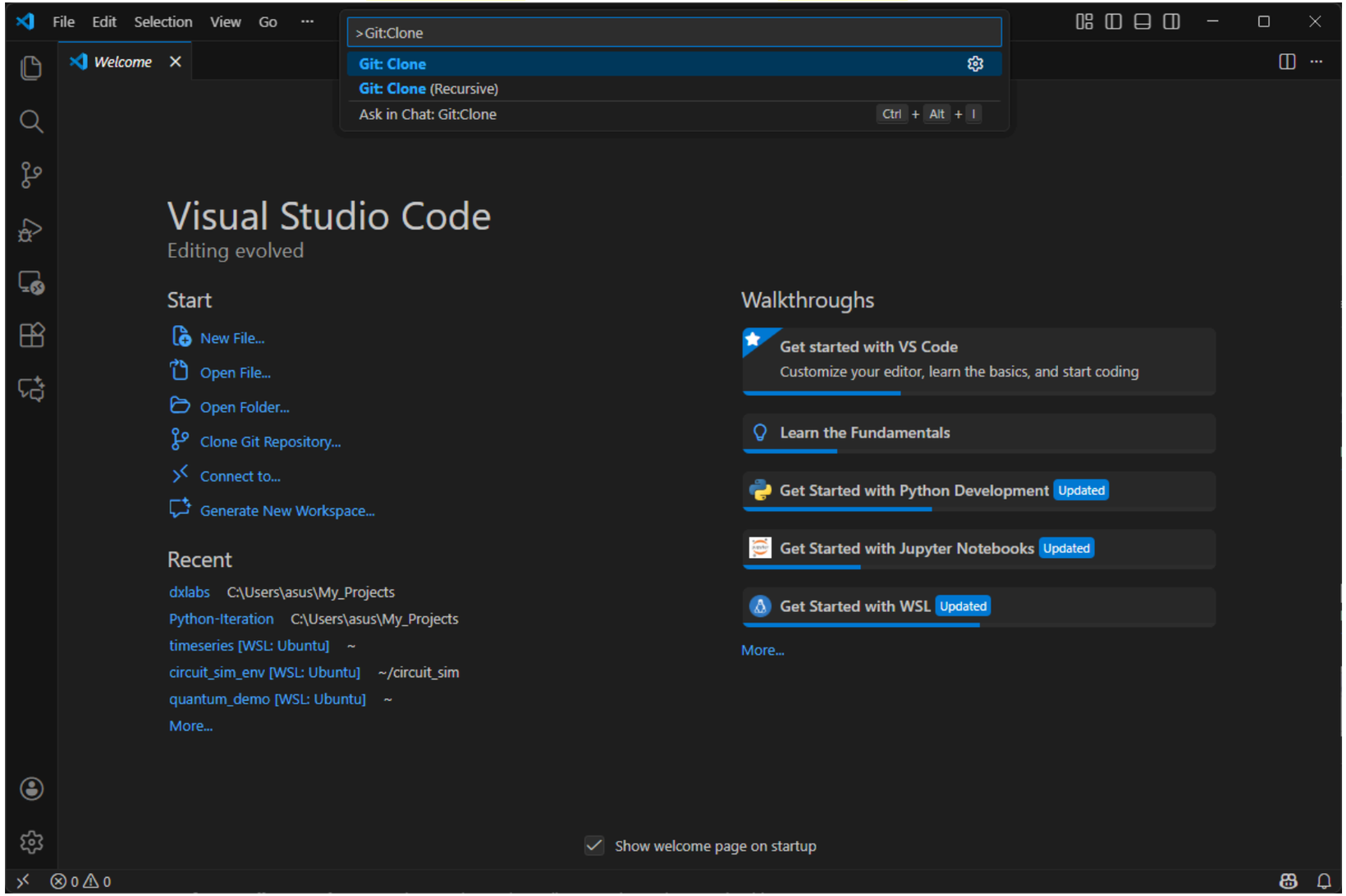
4. In the upper portion of VSCode, click on the `View` menu item.



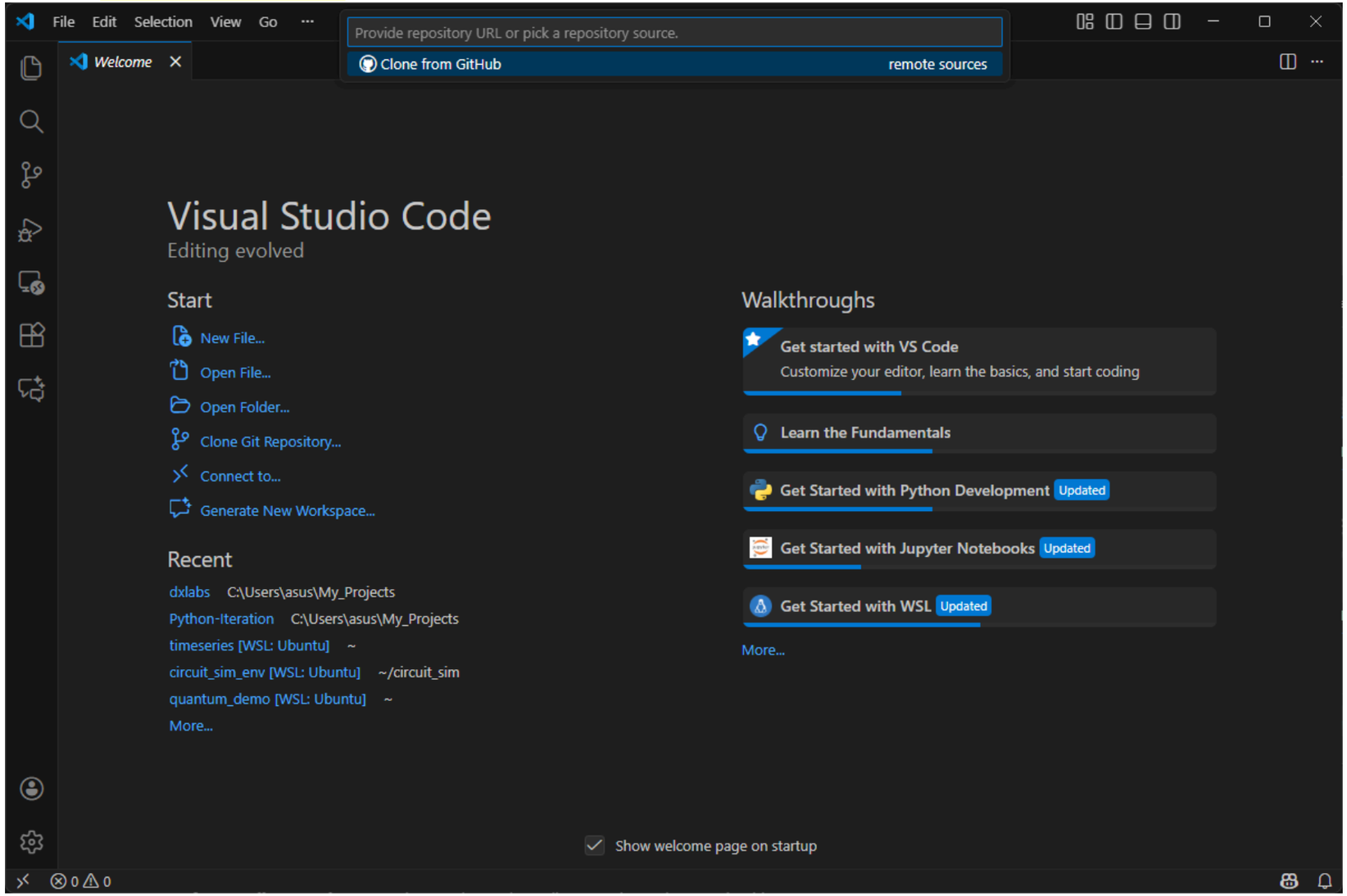
5. In the `View` menu, click on the `Command Palette` menu item.



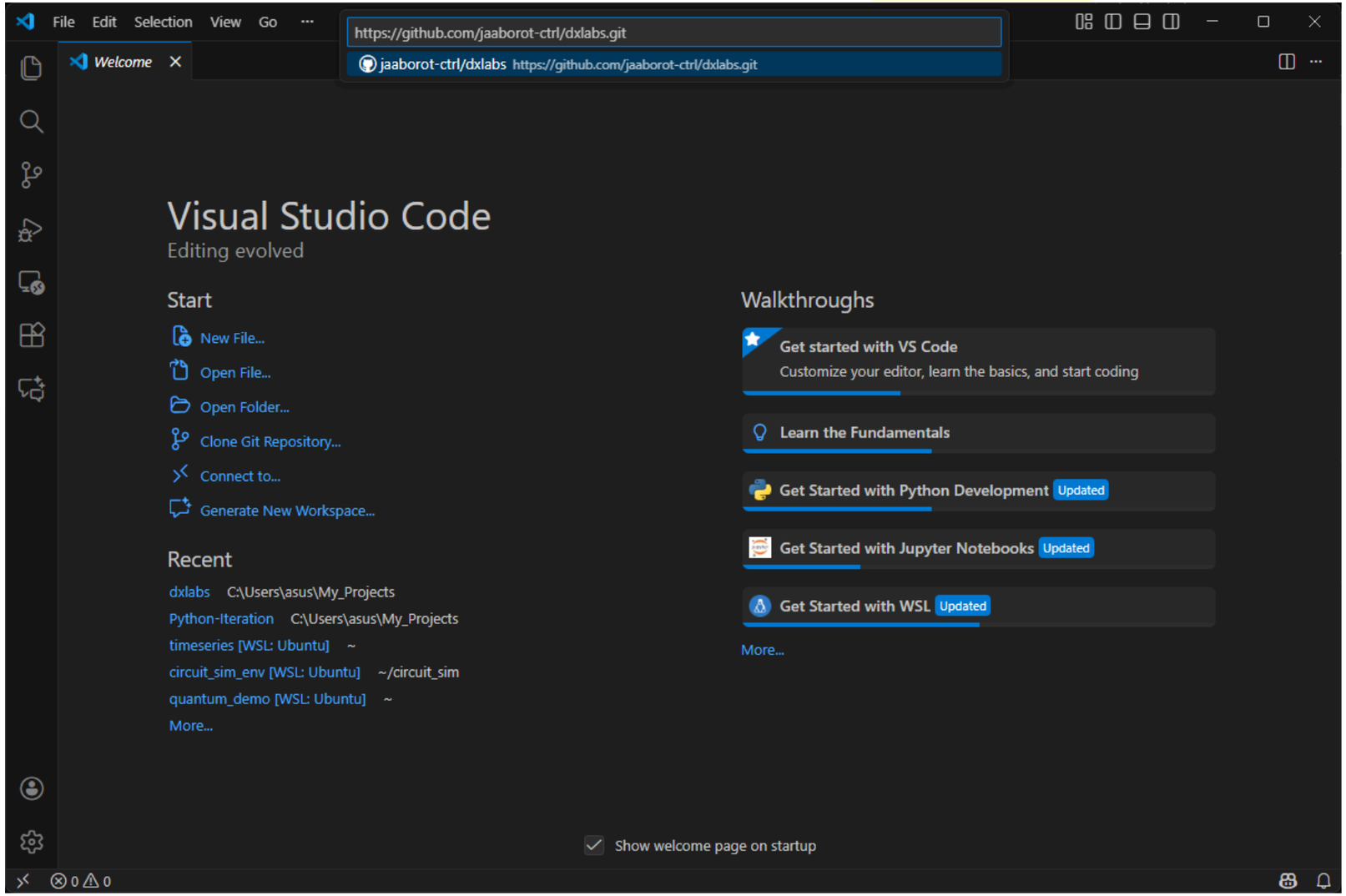
6. In the Command Palette tab, type `>Git: Clone` and click on the option `Git:Clone`.



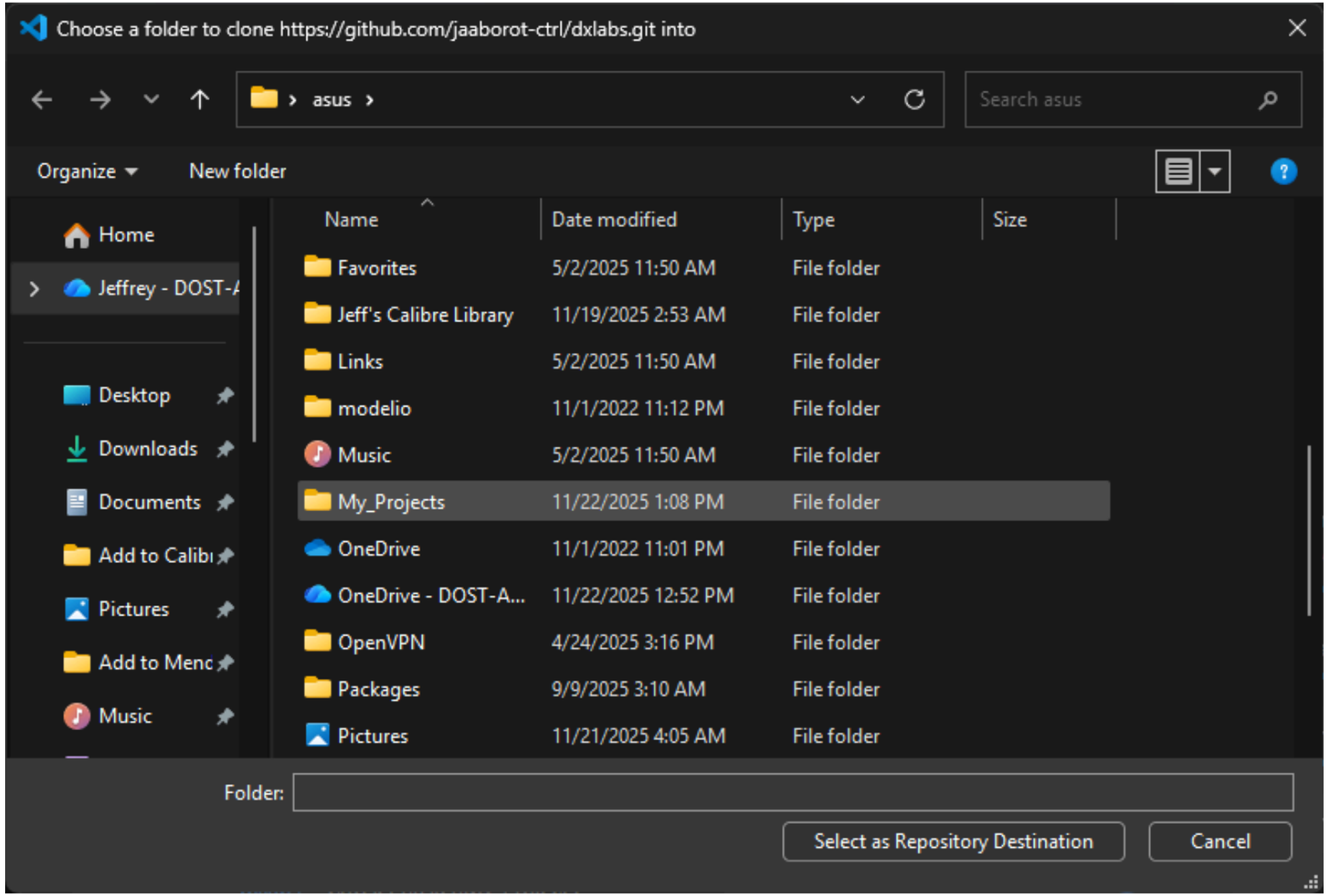
7. Click the `Clone from GitHub` menu item.



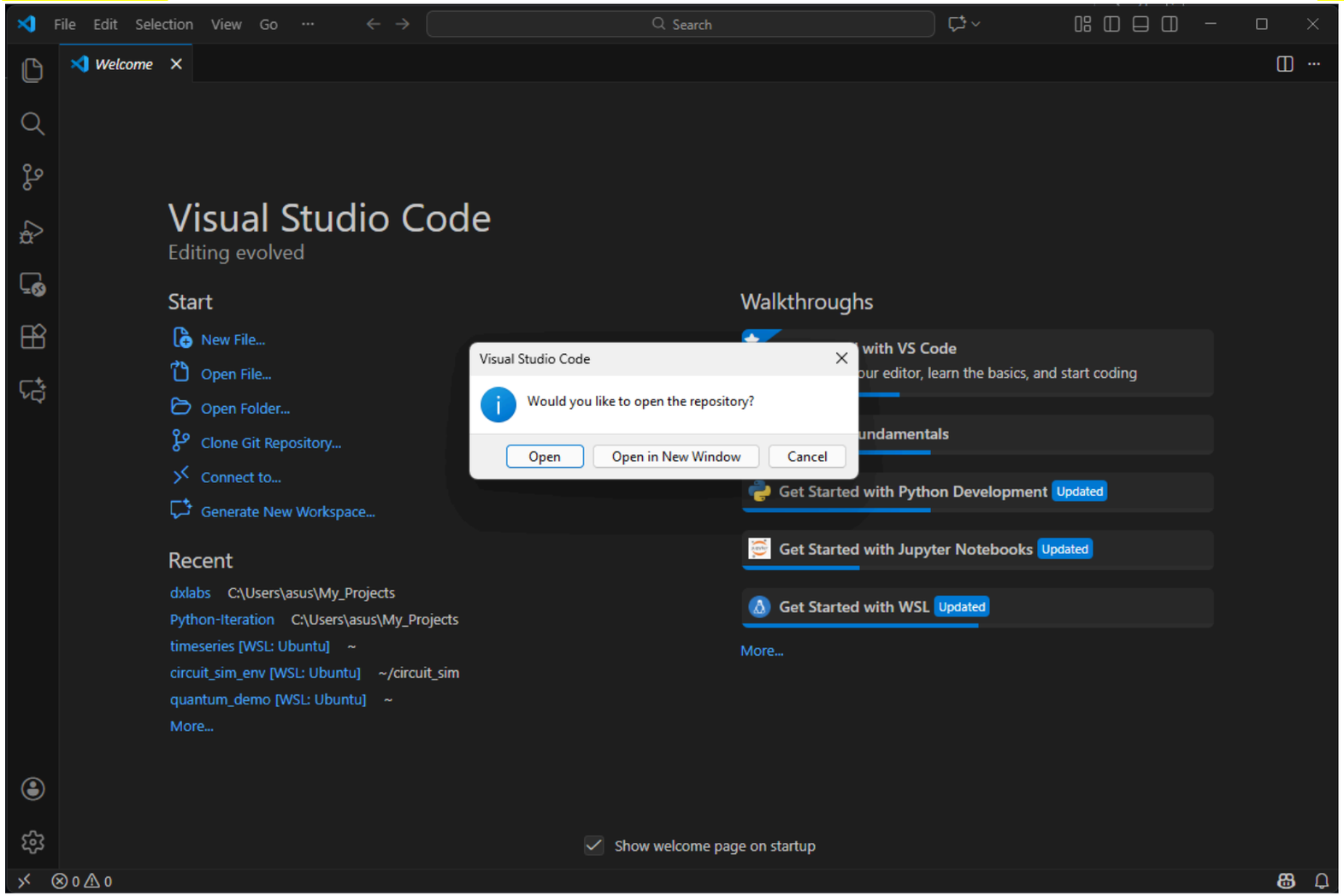
8. Paste the link of the forked `dxlab` repository from your GitHub account into the `Repository name` search box. Click on the option that will appear.



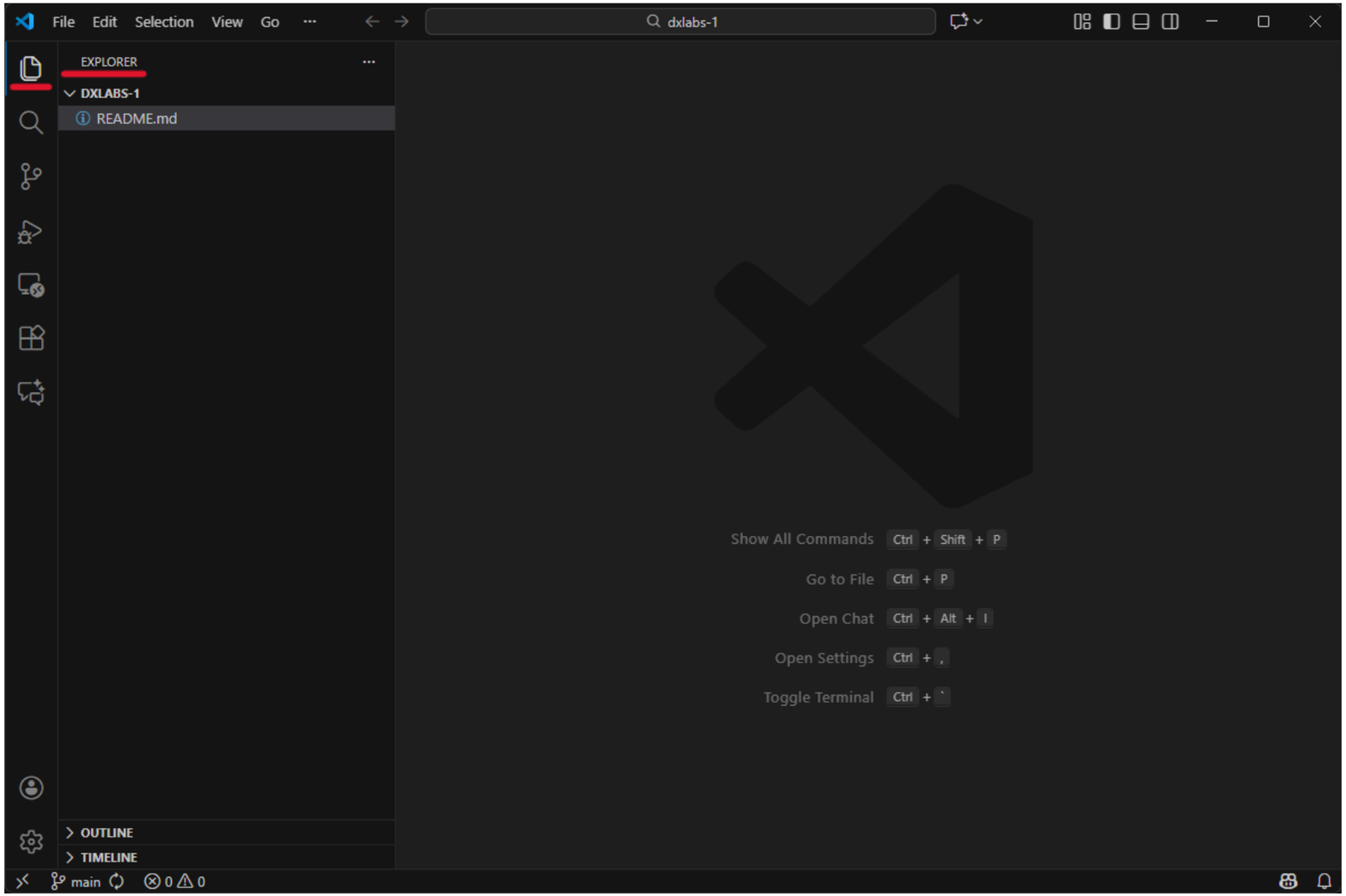
9. In the directory explorer, select the directory on your computer to which the copy of the forked `dxlab` repository will be downloaded.



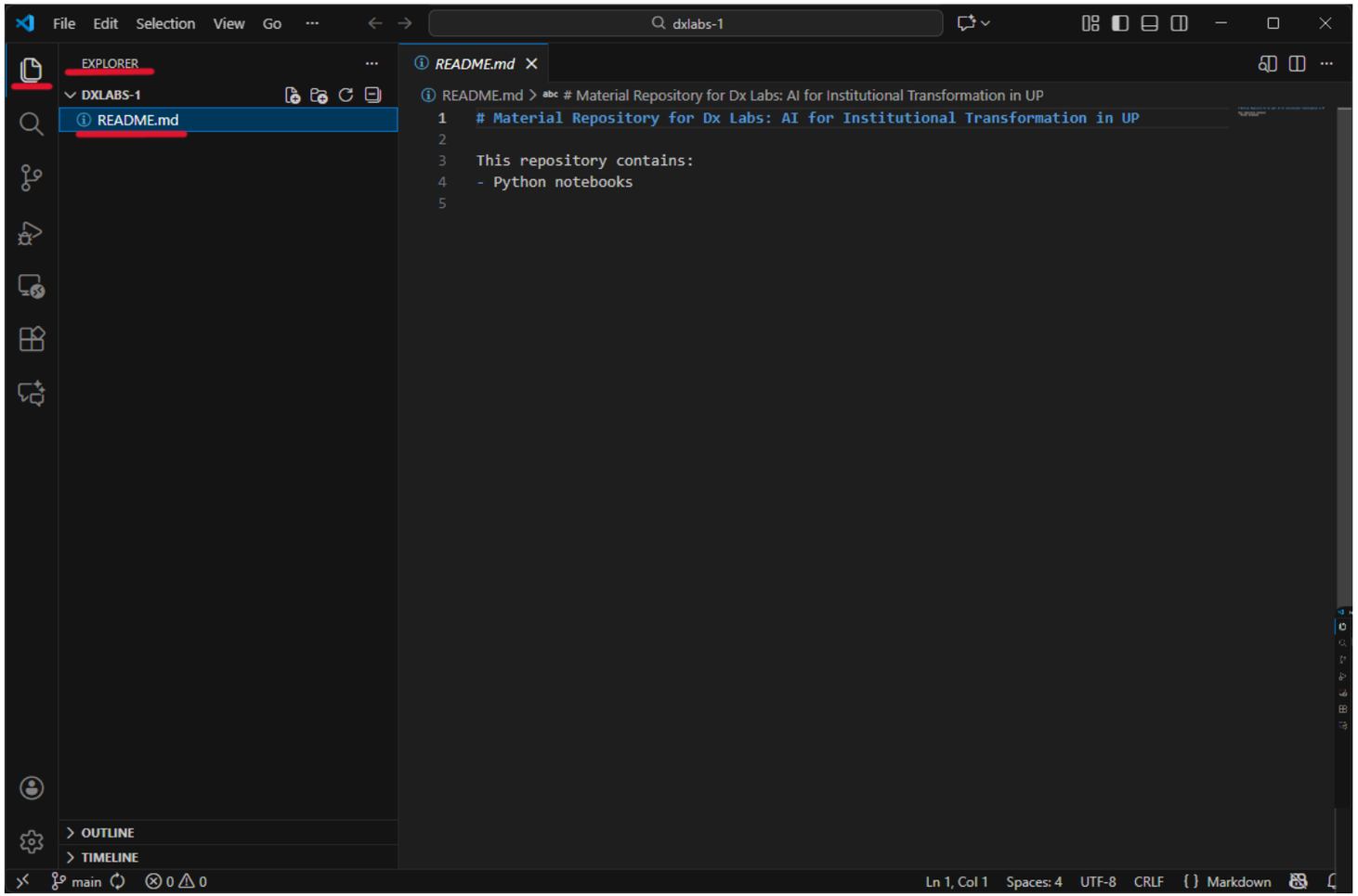
10. Once you have selected the local directory to which the `dxlab` code repository will be downloaded, click on the `Select as Repository Destination` button. A user input window will appear asking you if you would like to open the repository. Click on the `Open` button.



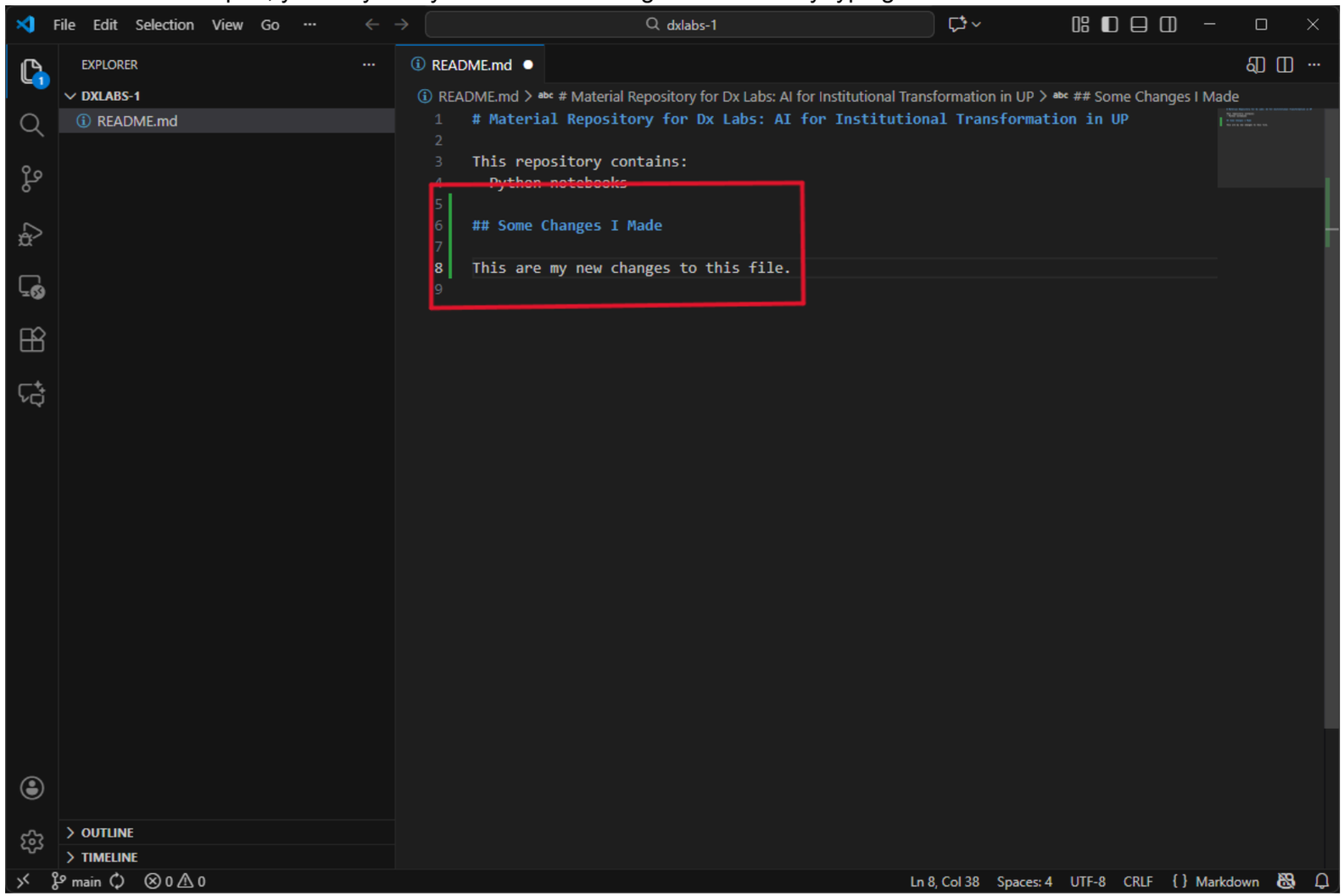
11. The downloaded copy of the forked `dxlab` code repository will be loaded into VSCode. You can now browse the downloaded files using the `Explorer` window on the left side of VSCode by clicking on each file. Clicking on a file will open it in a separate tab in VSCode.



3. **Adding Local Changes to a File in the Cloned Code Repository**
    1. To add changes to a file in your cloned code repository, click on that file from the Explorer window in VSCode. The contents of the file will be shown in a tab in VSCode.



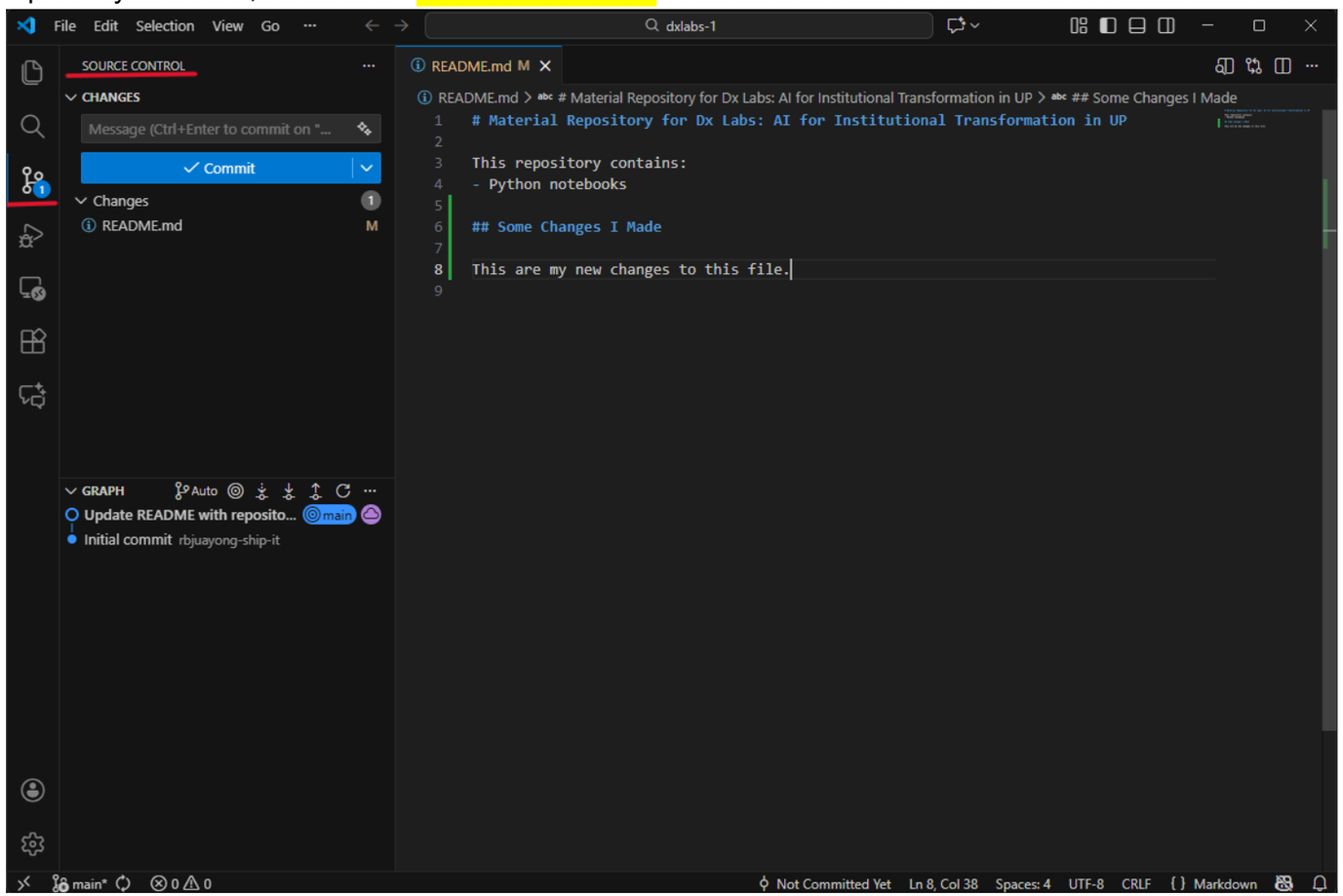
2. Now that the file is open, you may add your intended changes to the file by typing them into the content window in VSCode.



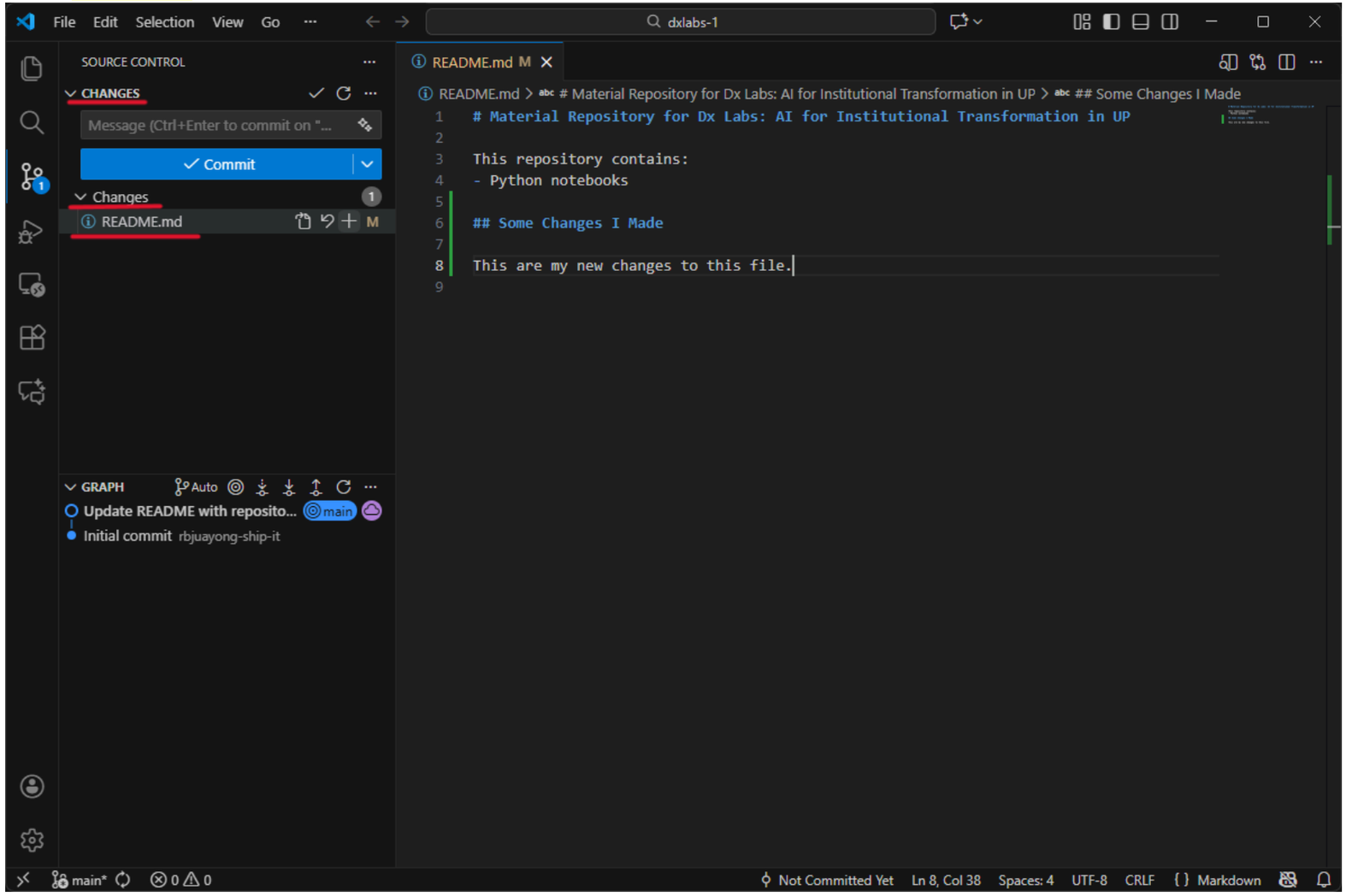
3. Once you have added any changes to the file, you will notice a green marker on the left-most end of each new line. This indicates that a new line has been added to the file and is a visualization of how Git keeps track of Add changes to a file.
4. Do not forget to save the changes that you have made to your file.

4. Reflecting Local Changes in a File to your Remote GitHub Repository

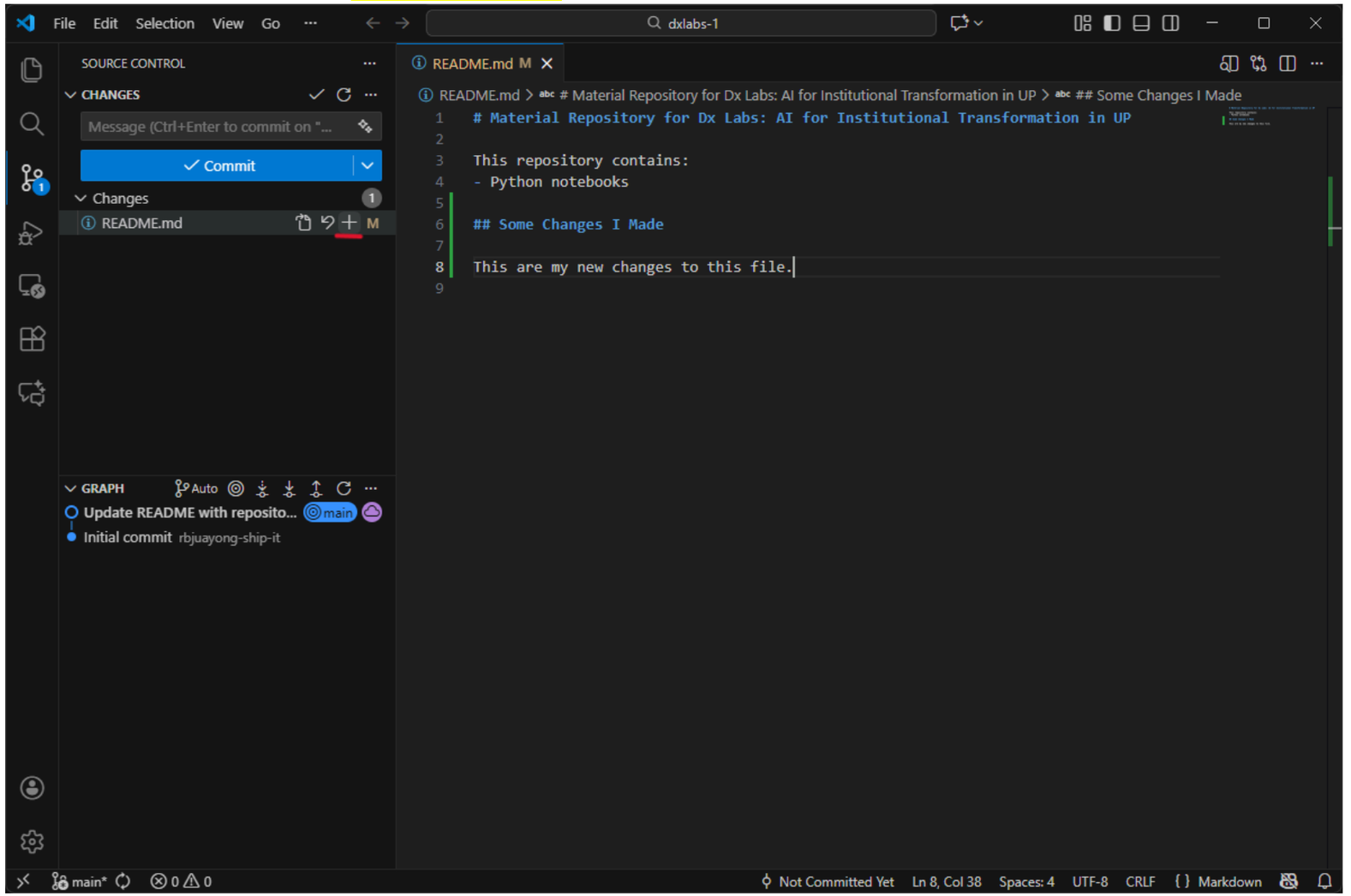
1. Once you have made your changes to a file in VSCode, you will need to reflect these changes to the corresponding file in your online GitHub repository. To do so, click on the `Source Control` icon on the left-most end of VSCode. This will show the Source Control window.

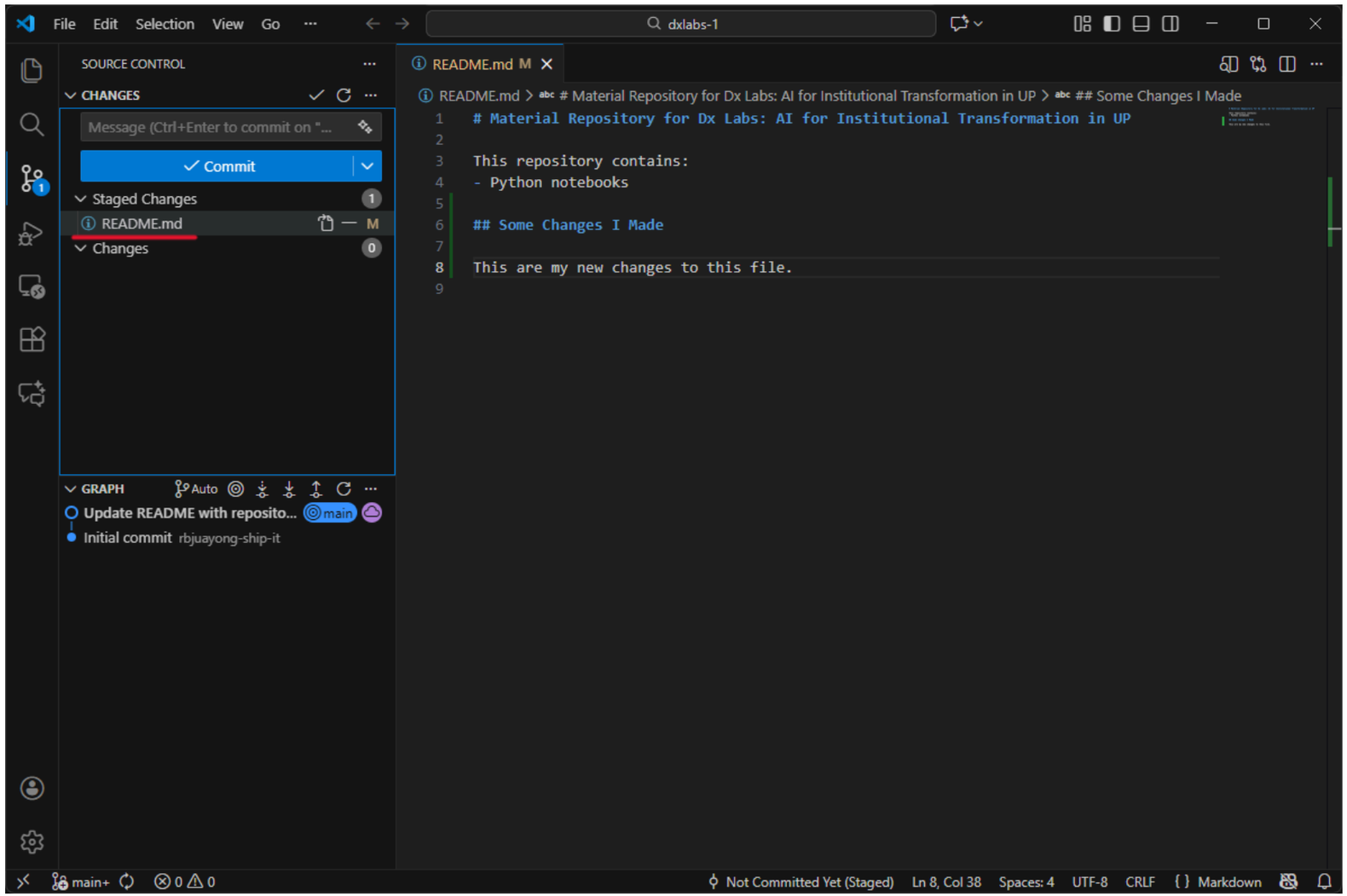


2. In the `Changes` subsection under the CHANGES section in the Source Control window, you will find the file that you have made changes to.



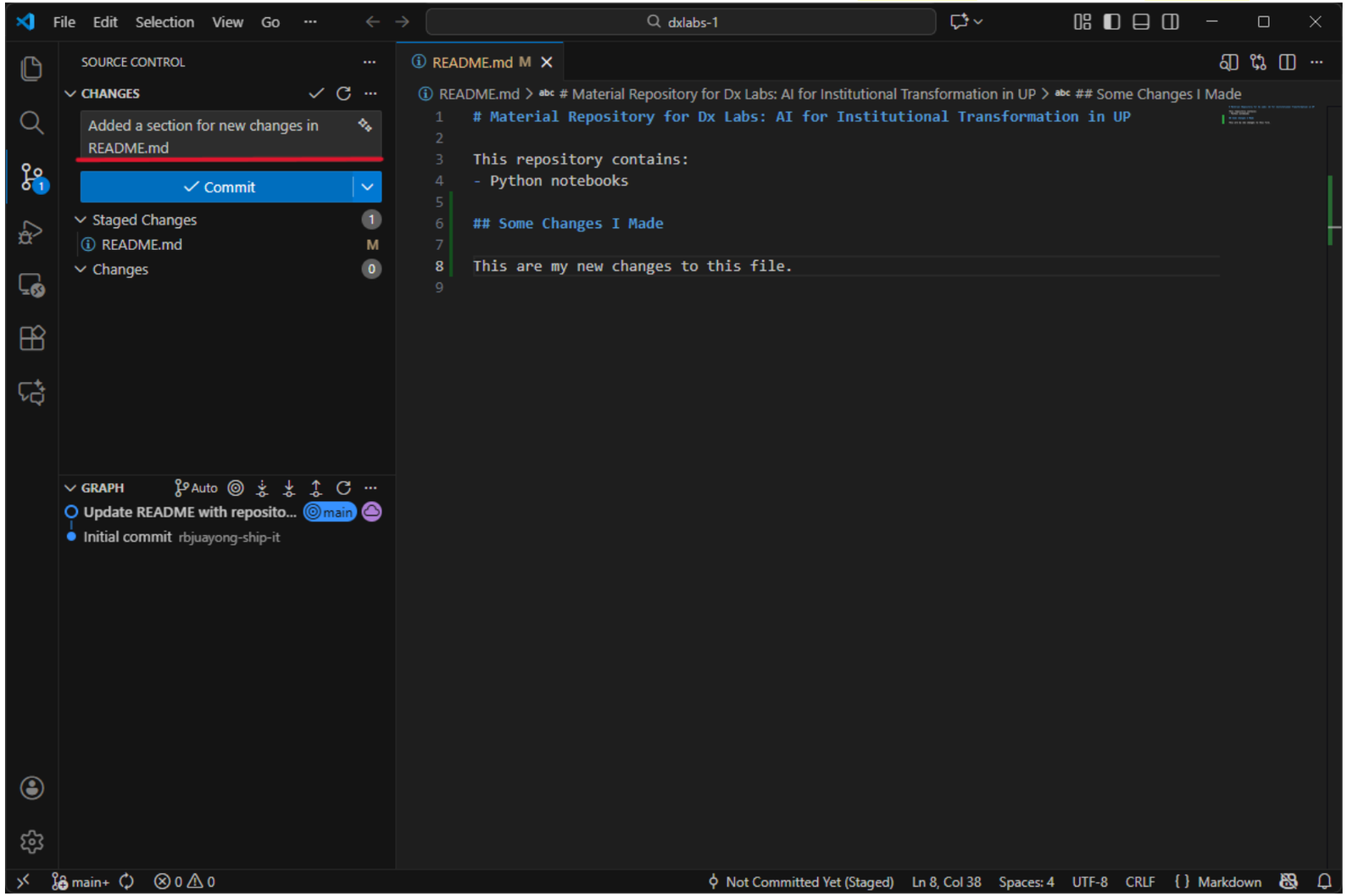
3. Click on the `Stage` button on the right side of the name of your file to indicate that you want your changes to be reflected in your GitHub repository. This will move your file into the `Staged Changes` subsection under the CHANGES section.



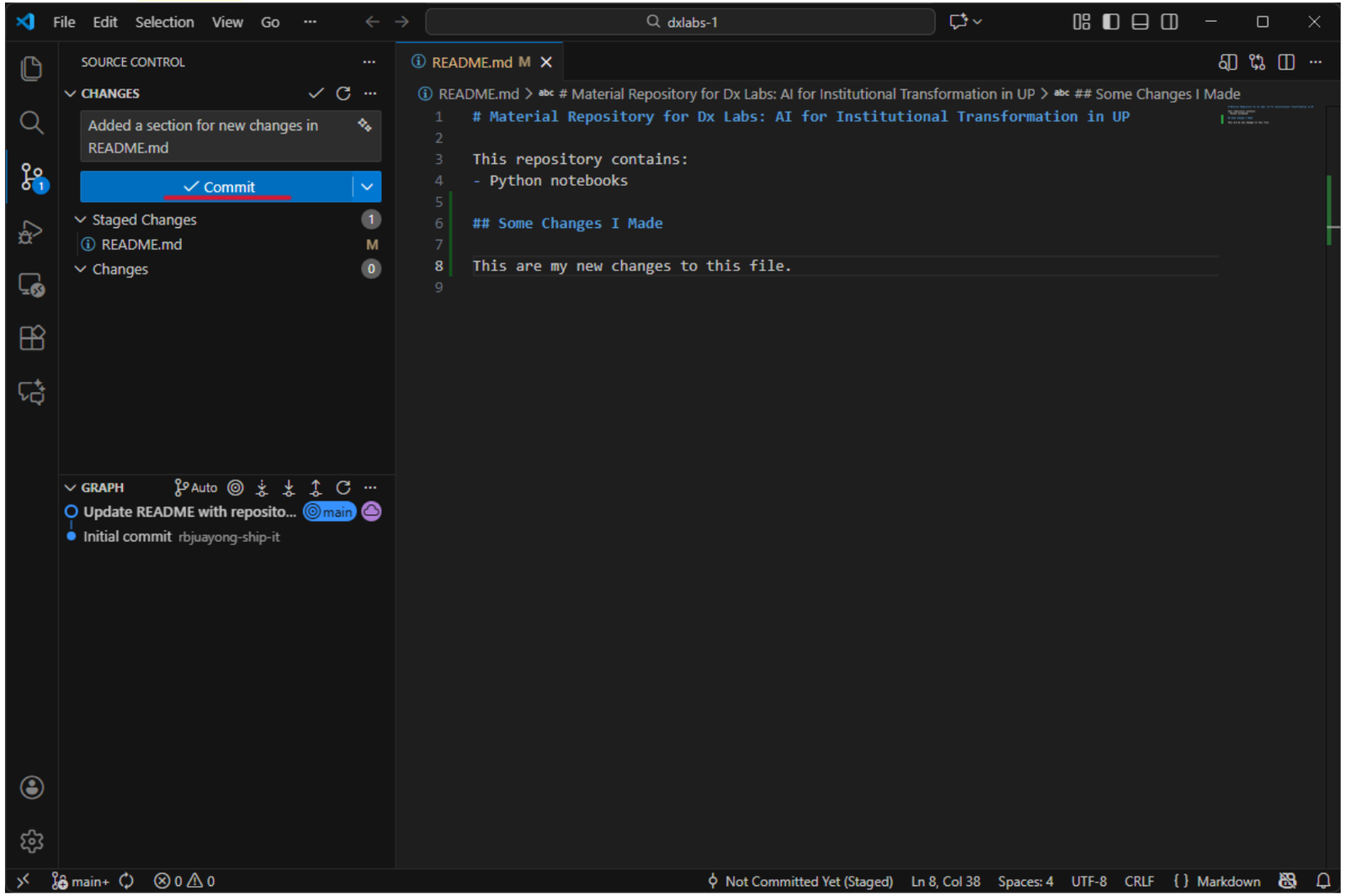


4. Once you have your file in the Staged Changes subsection, you can now proceed to commit your changes to the file. Committing changes you made in a file means that you are agreeing to have these changes reflected in the corresponding file in your GitHub repository.

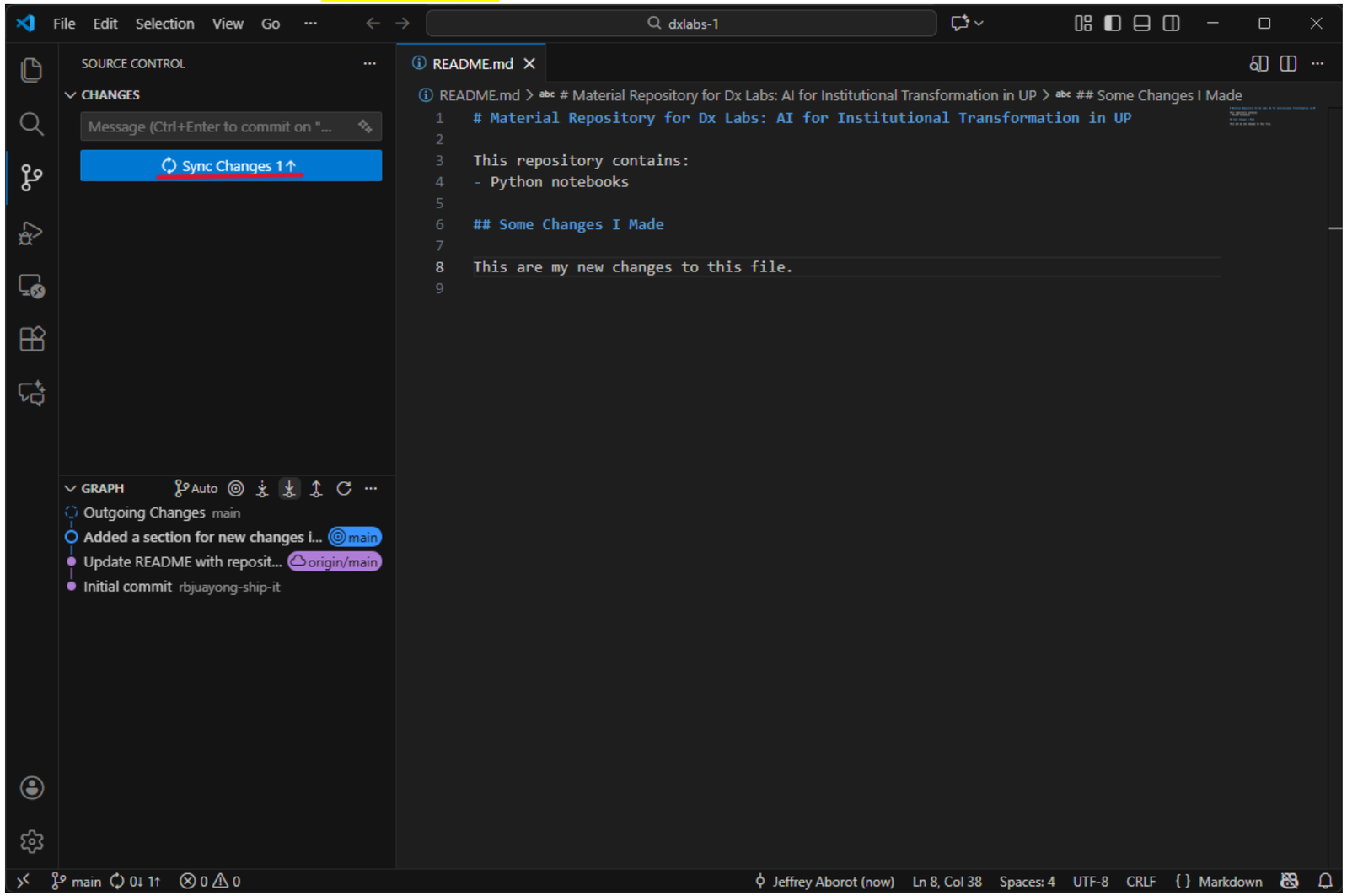
a. Key in a short description of the changes that you have made to the file in the `Message` field above the `Commit` button.



b. Click on the `Commit` button. This will prepare the changes to be reflected to the forked repository in your GitHub account.



c. Once you have committed the changes, you can now push these changes to its corresponding file in the forked repository in your online GitHub account. Click on the `Sync Changes` button to do so.



5. **Viewing the Changes in your GitHub Repository**
    1. To view the changes that you have pushed from your local copy of the dxlab code repository to your online GitHub repository, visit the page of the forked repository in your GitHub account and refresh the page. You will then see all the changes that you have made to the file that you have updated locally.

